

Multi-GPU Training

Multi-GPU training uses several GPUs to make a model train faster, fit a model that is too large for one device, or both. The central design question is **what gets replicated and what gets partitioned**: data, parameters, optimizer state, activations, or individual tensor operations.

Technique	Main thing distributed	Primary goal	Typical communication
DDP	Mini-batch	Throughput	Gradient all-reduce
FSDP	Parameters, gradients, optimizer state	Memory capacity plus throughput	Parameter all-gather and reduce-scatter
ZeRO	Optimizer state, gradients, then parameters by stage	Memory capacity	All-reduce or all-gather/reduce-scatter
Tensor parallelism	Individual tensor operations/layers	Fit and compute very large layers	All-reduce, all-gather, or reduce-scatter inside each layer
Pipeline parallelism	Consecutive layer groups	Fit deep models	Point-to-point activation/gradient transfers
Activation checkpointing	Saved forward activations over time	Reduce activation memory	Extra recomputation, normally no new network collective
Gradient accumulation	Micro-batches over time	Larger effective batch / lower peak activation memory	One optimizer update after several micro-batches

These methods are composable. A large production model may use tensor parallelism within a node, pipeline parallelism across layer groups, data parallelism across replicas, FSDP or ZeRO for state sharding, activation checkpointing for activations, and gradient accumulation to form the global batch.

Tensor Parallelism

1. Overview

Definition

Tensor parallelism (TP) partitions the computation and parameters of an individual layer across multiple GPUs. Rather than giving every GPU a full matrix multiplication, each GPU computes a slice—such as a subset of output columns or input rows—and collectives combine the partial results.

Why it matters

Some layers, attention heads, embedding tables, or logits are too large or too compute-intensive for one GPU. Tensor parallelism reduces per-GPU parameter and compute load for those layers and lets tightly connected GPUs cooperate on every layer.

Where it is used

- Large transformer attention and feed-forward layers
- Large vocabulary embeddings and output projections
- Mixture-of-experts components alongside expert parallelism
- Training and inference inside high-bandwidth GPU nodes

Why interviewers ask

TP tests matrix-shape reasoning, partition choices, collective placement, forward/backward dependencies, topology awareness, and the difference between storage sharding and sharded computation.

2. Core Idea

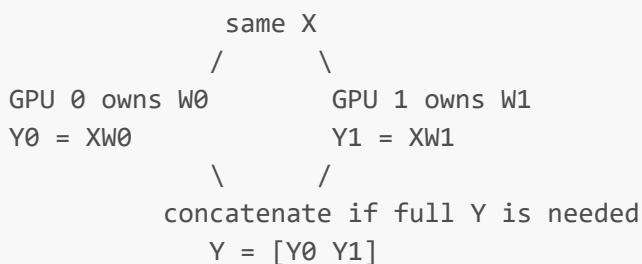
Intuition

Four workers must multiply a huge spreadsheet. Instead of each owning the whole spreadsheet, each worker owns selected columns or rows and computes that part. Sometimes their results already form separate pieces of the answer; sometimes partial sums must be added before the next operation.

Small example: column-parallel linear layer

For a linear layer $Y = XW$, split W by output columns:

$$W = [W_0 \ W_1], \quad Y = [XW_0 \ XW_1]$$



Each GPU stores half the weights and computes half the output features. If the next operation is also compatible with a sharded output, the concatenation can be delayed or avoided.

Small example: row-parallel linear layer

Split W by input rows and split X along its matching feature dimension:

$$X = [X_0 \ X_1], \quad W = \begin{bmatrix} W_0 \\ W_1 \end{bmatrix}, \quad Y = X_0 W_0 + X_1 W_1$$

Each GPU computes a partial Y ; an all-reduce sums the partial outputs.

Step-by-step execution

1. Choose a tensor dimension that makes local kernels large and balanced.
2. Partition parameters across a tensor-parallel process group.
3. Arrange the input as replicated or sharded to match the local matrix multiply.
4. Compute local output slices or partial sums.
5. Use all-gather, all-reduce, reduce-scatter, or all-to-all only where the next operator requires it.
6. Backward applies the transposed dependencies and communicates the necessary input/weight gradients.

3. Important Subtopics

Column parallelism

Splitting a weight matrix across output features produces independent output-feature shards. It needs no forward reduction; an all-gather is needed only if the next operation requires a full output. Backward input-gradient contributions from all column shards must be summed. Interview angle: identify which dimension corresponds to outputs under the stated matrix convention.

Row parallelism

Splitting across input features requires the input to be split the same way. Each rank produces a partial contribution to every output feature, so forward normally uses an all-reduce or reduce-scatter. Backward can naturally produce matching input-gradient shards.

Paired MLP partitioning

A transformer MLP commonly uses a column-parallel first projection and a row-parallel second projection:

```
replicated X
-> column-parallel W1
-> sharded hidden activation
-> elementwise activation locally
-> row-parallel W2
-> reduce partial outputs
-> replicated residual output
```

The intermediate remains sharded, avoiding an extra all-gather between the two matrices.

Attention-head partitioning

Query, key, and value projections can be split so each rank owns a subset of attention heads. Attention within a head is local; the output projection then combines head shards through a row-parallel pattern. The number of heads or key/value heads must be compatible with the TP size, or uneven/replicated handling is needed.

Vocabulary parallelism

Embedding rows or output vocabulary logits can be distributed across ranks. Token lookup requires selecting the owning shard and combining masked local results. Cross-entropy can be computed from sharded logits

by globally reducing maxima and exponential sums, avoiding materializing the full vocabulary logits.

Sequence parallelism

Some operations such as LayerNorm or dropout otherwise retain replicated activations within a TP group. Sequence parallelism partitions eligible activations along the sequence/token dimension and uses reduce-scatter/all-gather transitions. It complements tensor parallelism by reducing activation duplication.

Collective communication patterns

- **All-reduce:** sum partial results and replicate the sum.
- **All-gather:** reconstruct a full tensor from feature shards.
- **Reduce-scatter:** sum contributions while leaving the result sharded.
- **All-to-all:** redistribute slices, useful for certain sequence/context/expert layouts.

The best implementation fuses or pairs collectives so tensors stay sharded as long as consumers allow.

Topology-aware grouping

TP communicates inside almost every transformer layer, so it prefers the fastest links—NVLink/NVSwitch within a node. Data parallelism, which communicates less frequently, is often placed across slower node boundaries. A logically correct but topology-blind mapping can perform poorly.

Tensor-parallel degree

More TP ranks reduce per-rank layer state and arithmetic, but increase collective participants and shrink local GEMMs. Eventually communication and poor kernel utilization dominate. The degree is constrained by tensor divisibility, node topology, and the minimum efficient matrix size.

4. Real-World Example

Consider a transformer MLP with hidden size h and expansion size $4h$ on two GPUs:

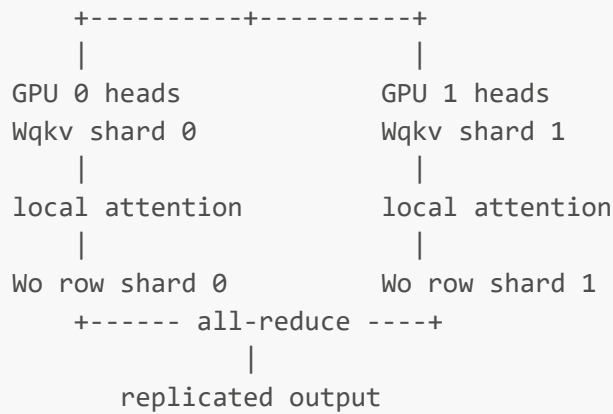
```
W1 shape: h x 4h, split columns -> each GPU owns h x 2h
GeLU: applied independently to each GPU's 2h features
W2 shape: 4h x h, split rows -> each GPU owns 2h x h
Each GPU computes partial h output
All-reduce adds partial outputs before the residual connection
```

Only one main reduction is required for this pair, and neither GPU stores the full MLP weights. In a production transformer, attention projections use a similar head-aligned partition.

5. Diagrams / Mental Models

```
One transformer block, TP = 2

    replicated residual stream X
        |
```



Mental model: **split one layer sideways; communicate at algebraic boundaries.**

Partition	Local result	Typical forward communication
Weight by output columns	Output-feature shard	All-gather only if full result needed
Weight by input rows	Partial sum for full output	All-reduce or reduce-scatter
Attention by heads	Head subset	Combine at output projection
Vocabulary by rows	Vocabulary shard	Specialized lookup/loss reductions

6. Common Interview Questions

Q1. What is tensor parallelism?

Answer: It partitions individual tensor operations and their parameters across GPUs so ranks jointly execute the same layer on the same logical batch.

Interviewer expects: within-layer computation sharding.

Common mistake: Calling any parameter sharding tensor parallelism.

Q2. How does TP differ from data parallelism?

Answer: Data parallel ranks run complete model replicas on different samples and combine gradients. TP ranks cooperate on the same layer, each holding and computing a tensor slice, with communication inside forward and backward.

Interviewer expects: different data versus same logical operation.

Common mistake: Saying both merely divide the batch.

Q3. Explain column-parallel matrix multiplication.

Answer: Split the weight by output columns. Every rank multiplies the same input by its local columns and obtains a disjoint output-feature shard. Concatenate/all-gather only when a consumer needs the full output.

Interviewer expects: correct shapes and no required forward summation.

Common mistake: All-reducing disjoint columns, which would be algebraically wrong.

Q4. Explain row-parallel matrix multiplication.

Answer: Split the weight and input along the contracted input-feature dimension. Each rank computes a partial contribution to all output features, and the contributions are summed.

Interviewer expects: input split and output reduction.

Common mistake: Concatenating partial sums rather than adding them.

Q5. Why pair column- and row-parallel layers?

Answer: The first layer's output-feature shard is exactly the input-feature shard required by the second. Elementwise activation stays local, and only the second layer's partial outputs need reduction.

Interviewer expects: avoid an intermediate all-gather.

Common mistake: Materializing the expanded hidden activation on every rank.

Q6. Why is TP usually kept within a node?

Answer: Its collectives occur in nearly every layer and can lie directly on the critical path. High-bandwidth, low-latency NVLink/NVSwitch is better suited than inter-node networking.

Interviewer expects: communication frequency and topology.

Common mistake: Choosing process groups without considering physical links.

Q7. Does increasing TP degree always speed training?

Answer: No. Local GEMMs become smaller, collective cost rises, and divisibility constraints grow. TP is often used for memory/capacity first; speed depends on sufficient compute per rank and fast links.

Interviewer expects: strong-scaling limit.

Common mistake: Assuming perfect linear speedup.

Q8. How is LayerNorm handled?

Answer: It depends on what dimension is sharded. If the normalized hidden dimension is partitioned, global mean and variance need reductions; many designs instead keep the residual stream replicated or use sequence parallelism so each token's hidden vector is local.

Interviewer expects: statistics depend on the normalized dimension.

Common mistake: Saying LayerNorm is always communication-free.

Q9. How can cross-entropy work with sharded vocabulary logits?

Answer: Compute local maximum, reduce to the global maximum, compute local shifted exponential sum, reduce the denominator, and retrieve the target logit from its owning shard. This yields exact softmax loss without gathering all logits.

Interviewer expects: numerically stable global max and sum reductions.

Common mistake: Averaging local softmax probabilities.

Q10. What determines a legal TP degree?

Answer: Relevant dimensions—hidden size, attention heads, key/value heads, and vocabulary partitions—must be divisible or explicitly support uneven splits. The degree must also match memory, topology, and

efficient kernel shapes.

Interviewer expects: both algebraic and hardware constraints.

Common mistake: Checking only the GPU count.

7. Deep-Dive Questions

Q1. Derive backward communication for a column-parallel layer.

For $Y_i = XW_i$, each rank gets dY_i . It computes local $dW_i = X^T dY_i$ without communication and a partial $dX_i = dY_i W_i^T$. Because all column partitions contribute to the same dX , ranks sum dX_i , typically with all-reduce or a compatible reduce-scatter layout.

Q2. When can an all-reduce be replaced by reduce-scatter?

When the consumer can accept a sharded result. Reduce-scatter performs the reduction but avoids replicating the output, reducing per-rank bytes and activation memory. A later all-gather may still be needed, so the full operator chain must be optimized rather than one collective in isolation.

Q3. Why can TP hurt arithmetic efficiency?

Partitioning shrinks GEMM dimensions. Small or awkward matrices use Tensor Cores and memory bandwidth less efficiently, while collective synchronization becomes a larger fraction of step time. Padding for divisibility can also waste arithmetic.

Q4. How do rotary embeddings and attention masks interact with head parallelism?

Rotary position transforms and masks are usually applied independently to local query/key heads, so they need no cross-rank communication if required positional data is locally available. Grouped-query attention needs careful mapping because fewer key/value heads may be shared among several query heads.

Q5. How would you map 3D parallelism onto 64 GPUs?

Choose tensor-parallel groups along the fastest intra-node links, assign consecutive layer partitions to pipeline stages, and use remaining replicas as data-parallel groups. For example, $TP=8, PP=4, DP=2$ gives $8 \times 4 \times 2 = 64$; exact axes depend on eight-GPU node topology and model dimensions.

8. Comparison Tables

Tensor versus pipeline versus data parallelism

Aspect	Data parallel	Tensor parallel	Pipeline parallel
Split	Batch/examples	Operations/tensor dimensions	Consecutive layer groups
Model per rank	Full	Slices of most layers	Full layers for one stage
Communication	Gradients per step	Activations/partials inside layers	Activations between stages

Aspect	Data parallel	Tensor parallel	Pipeline parallel
Best topology	Can span nodes	Fastest local links	Moderate bandwidth, point-to-point
Main inefficiency	Gradient synchronization	Frequent collectives/small GEMMs	Pipeline bubbles

Column versus row parallel

Aspect	Column parallel	Row parallel
Weight split	Output dimension	Input/contracted dimension
Input	Usually replicated	Sharded
Local output	Disjoint feature shard	Partial full-feature sum
Forward combine	Optional all-gather	Sum via all-reduce/reduce-scatter
Natural pairing	First MLP/QKV projection	Second MLP/output projection

9. Common Mistakes

- Confusing parameter storage sharding with operation sharding.
- Mixing row/column conventions without writing matrix shapes.
- Summing column shards or concatenating row partial sums.
- Gathering intermediate activations unnecessarily.
- Ignoring backward communication.
- Assuming attention heads are always divisible by TP degree.
- Spanning slow links with a communication-heavy TP group.
- Increasing TP until local matrix multiplications are inefficient.
- Forgetting global statistics for a normalization dimension that is sharded.
- Assuming TP alone creates different data batches on each rank.

10. Edge Cases / Special Cases

- **Prime or awkward hidden dimensions:** uneven shards or padding may be necessary.
- **Grouped-query/multi-query attention:** key/value head count can constrain TP more than query heads.
- **Tiny batches during inference:** communication latency can dominate even though memory requires TP.
- **Residual connections:** operands must have compatible replicated/sharded layouts.
- **Dropout RNG:** shards need deterministic nonoverlapping random streams when reproducibility matters.
- **Bias terms:** a row-parallel output bias must be added after reduction, not independently before sum unless compensated.
- **Embedding ownership:** out-of-shard token IDs require masking/routing and a combine operation.
- **Quantization:** scales and group boundaries must align with tensor partitions or be communicated.

11. How to Explain in Interview

Tensor parallelism splits individual layers across GPUs. For a linear layer, a column split gives each rank different output features, while a row split gives partial sums that must be reduced. Transformer implementations pair these layouts so intermediate activations stay sharded and communicate only at necessary algebraic boundaries. TP helps very large layers fit but requires fast links because communication happens inside almost every layer.

12. Quick Revision Notes

- **Column split:** output shards; gather only if required.
- **Row split:** partial sums; reduce them.
- **Transformer pattern:** column first, elementwise activation local, row second, one reduction.
- **Attention:** usually partition heads.
- **Topology:** TP on fastest links.
- **Scaling trap:** more ranks mean smaller GEMMs and more communication.
- **Core distinction:** TP shards computation; ZeRO/FSDP mainly shard state over its lifetime.

13. Practice Tasks

1. For $X[32, 4096]$ and $W[4096, 16384]$, write local shapes under four-way column parallelism.
2. Derive forward and backward communication for both row and column splits.
3. Implement a two-process column-parallel linear layer and compare output/gradients with a full layer.
4. Draw a tensor-parallel transformer MLP with exactly one forward reduction.
5. Design stable vocabulary-parallel cross-entropy without gathering logits.
6. Benchmark matrix multiplication efficiency as the TP degree shrinks local dimensions.
7. Map TP groups to an eight-GPU NVLink topology.
8. Explain how a residual add constrains tensor layouts.

14. Final Cheat Sheet

Item	Tensor-parallel answer
Core definition	GPUs jointly compute slices of each layer
Why it matters	Fits and accelerates individual huge layers
Column split	Disjoint output features
Row split	Partial outputs that must be summed
Transformer trick	Pair column then row layouts to avoid an intermediate gather
Main comparison	TP splits operations; PP splits layers; DP splits data
Main risk	Frequent collectives overwhelm smaller local kernels
One-line answer	"Tensor parallelism partitions a layer's matrix operations across GPUs and combines partial results at required boundaries."

Pipeline Parallelism

1. Overview

Definition

Pipeline parallelism (PP) partitions a model by depth: each device or device group owns a consecutive set of layers called a **stage**. A micro-batch flows from early to later stages during forward, then gradients flow backward in reverse.

Why it matters

Storing only some layers per stage makes a deep model fit across aggregate GPU memory. Dividing a large batch into micro-batches allows different stages to work simultaneously, increasing utilization compared with processing one whole batch through one stage at a time.

Where it is used

- Very deep transformer and neural-network training
- Multi-node large-model systems combined with TP and data parallelism
- Inference pipelines where stages stream requests or token work
- Systems whose model layers fit individually but total depth does not fit one device

Why interviewers ask

PP makes scheduling visible. Interviewers can test pipeline bubbles, micro-batching, activation lifetime, load balance, point-to-point communication, weight-version consistency, and the difference between latency and throughput.

2. Core Idea

Intuition

An assembly line has stations for cutting, painting, and packing. Processing one item through all stations sequentially leaves most workers idle. Feed smaller items one after another, and after warmup every station works on a different item. The line is only as fast as its slowest station.

Small example

Two stages and four micro-batches:

```
time ->      1      2      3      4      5
stage 0:     F0     F1     F2     F3     idle
stage 1:     idle   F0     F1     F2     F3
```

For training, backward work must also be scheduled:

```
activation: stage 0 --send--> stage 1
gradient:  stage 0 <--send-- stage 1
```

Step-by-step execution

1. Partition ordered layers into stages with roughly balanced work and memory.
2. Split the mini-batch into m micro-batches.
3. Stage 0 forwards micro-batch 0 and sends its boundary activation to stage 1.
4. Stage 0 begins micro-batch 1 while stage 1 processes micro-batch 0.
5. The last stage computes loss and begins backward.
6. Activation gradients travel back stage by stage.
7. After all micro-batches contributing to the batch complete, each stage applies a consistent optimizer update.

For p equally timed stages and m micro-batches under a simple all-forward-then-all-backward schedule, the idealized bubble fraction is roughly:

$$\frac{p - 1}{m + p - 1}$$

for each directional pipeline. Larger m amortizes fill/drain bubbles but changes overhead and activation memory.

3. Important Subtopics

Stage partitioning

Equal layer counts do not guarantee balance: attention, MLP, embeddings, output loss, and different sequence lengths have different cost and memory. Pipeline throughput is limited by the slowest stage, so partition by measured time and memory while minimizing large boundary activations.

Micro-batches and chunks

A batch is divided into smaller micro-batches to keep stages busy. More micro-batches reduce bubble fraction but launch smaller kernels, add communication messages, and may increase saved activation count. The optimizer still normally steps once per complete global batch.

GPipe schedule

GPipe runs forward for all micro-batches, then backward for all in reverse. It is conceptually simple and uses one weight version for the batch, but stores many outstanding activations unless activation checkpointing is used.

1F1B schedule

After warmup, **one-forward-one-backward (1F1B)** alternates forward and backward micro-batches. It limits the number of live activations compared with all-forward/all-backward and keeps the pipeline active. A flush between batches preserves synchronous weight semantics.

Interleaved pipeline

A physical GPU can own multiple nonconsecutive virtual stages or model chunks. Interleaving reduces bubble time by making stage spacing finer, but adds schedule complexity and more communication. It is useful when memory and topology permit.

Pipeline bubble

During fill, later stages wait; during drain, earlier stages wait. The bubble grows with stage count and stage imbalance, and shrinks relative to useful work with more micro-batches. It cannot be discussed only as an abstract formula—the slowest stage and communication gaps matter.

Activation and gradient communication

PP mainly uses point-to-point sends/receives between adjacent stages. Boundary activation size, dtype, sequence length, and link bandwidth determine transfer time. Backward sends a gradient tensor of a similar boundary shape in reverse.

Activation memory

Each stage must retain information for micro-batches whose backward has not arrived. GPipe can hold many such activations; 1F1B bounds them more tightly. Activation checkpointing trades recompute for less stored information.

Weight consistency

If weights update while old micro-batches remain in flight, different micro-batches may use different versions, producing stale or inconsistent gradients. Flush schedules update only after the batch drains. Asynchronous schedules need weight stashing or accept altered optimization semantics.

Loss and skip connections

The final stage commonly computes loss. Long skip connections crossing stage boundaries require additional activation transfers or careful partition placement. Transformer residuals usually remain within a block, making block boundaries convenient stage cuts.

4. Real-World Example

A 48-block transformer is assigned to four stages, 12 blocks each. Tensor parallelism may use four GPUs inside every stage, making 16 GPUs per pipeline replica. A training batch is divided into 32 micro-batches and scheduled with 1F1B. Activations cross only three stage boundaries; after all micro-batches complete, data-parallel replicas synchronize their stage-local gradients.

```
DP replica A: [TP group: blocks 0-11] -> [12-23] -> [24-35] -> [36-47]
DP replica B: [TP group: blocks 0-11] -> [12-23] -> [24-35] -> [36-47]
               stage-local replicas synchronize across the DP axis
```

This is hybrid or 3D parallelism: TP within a stage, PP across stages, and DP across pipeline replicas.

5. Diagrams / Mental Models

1F1B-style steady state, illustrative

```

time ->   t0    t1    t2    t3    t4    t5    t6
stage 0:  F0    F1    F2    B0    F3    B1    ...
stage 1:           F0    F1    B0    F2    B1    ...
stage 2:                   F0    B0    F1    B1    ...
           warmup |<-- alternating -->| drain

```

Mental model: **split layers vertically, stream micro-batches through stages, and pay fill/drain bubbles.**

Scheduling goal	Useful knob	Trade-off
Smaller bubble	More micro-batches	Smaller kernels/more messages
Lower activation peak	1F1B or checkpointing	Schedule or recompute cost
Better balance	Repartition stages	May increase boundary traffic
Fewer weight-version issues	Flush before update	Leaves fill/drain bubble

6. Common Interview Questions

Q1. What is pipeline parallelism?

Answer: It assigns different consecutive layer groups to different stages and streams micro-batches through them, sending activations forward and activation gradients backward.

Interviewer expects: depth partition plus micro-batch pipeline.

Common mistake: Describing different GPUs processing different data through full replicas.

Q2. Why are micro-batches needed?

Answer: Without them, only one stage works at a time for a single batch. Micro-batches allow stages to operate concurrently on different pieces of the batch and amortize fill/drain idle time.

Interviewer expects: concurrency and bubble reduction.

Common mistake: Assuming every micro-batch performs its own optimizer update.

Q3. What is a pipeline bubble?

Answer: Idle stage time during pipeline fill, drain, imbalance, or communication stalls. With equal stages, more micro-batches reduce its fraction, though not necessarily total latency.

Interviewer expects: fill/drain plus imbalance.

Common mistake: Saying bubbles arise only at startup of the whole training job.

Q4. Compare GPipe and 1F1B.

Answer: GPipe runs all forwards then all backwards, which is simple but keeps more activations live. 1F1B alternates forward and backward after warmup, reducing outstanding activations while retaining a pipeline flush for synchronous updates.

Interviewer expects: schedule and memory difference.

Common mistake: Claiming 1F1B eliminates all bubbles.

Q5. How do you choose stage boundaries?

Answer: Balance measured compute and memory, keep parameter-heavy or expensive operations distributed sensibly, and avoid boundaries that transfer huge or multiple tensors. Respect skip connections and topology.

Interviewer expects: time, memory, and communication, not just layer count.

Common mistake: Dividing the number of layers equally without profiling.

Q6. What communicates between stages?

Answer: Forward boundary activations and, during backward, gradients with respect to those activations. Parameters remain on their owning stage unless another sharding dimension is added.

Interviewer expects: point-to-point traffic.

Common mistake: Saying every stage all-reduces activations with every other stage.

Q7. Does PP reduce activation memory?

Answer: It reduces the number of layers resident per stage but can retain activations for multiple in-flight micro-batches. Depending on schedule, activation memory may still dominate; 1F1B and checkpointing help.

Interviewer expects: distinguish layer count from in-flight count.

Common mistake: Assuming activations divide exactly by stage count.

Q8. What limits pipeline throughput?

Answer: The slowest stage's compute plus exposed communication determines steady-state cadence. Fill/drain bubbles and load imbalance reduce overall utilization.

Interviewer expects: bottleneck-stage principle.

Common mistake: Averaging stage times instead of considering the maximum.

Q9. How does PP preserve optimizer correctness?

Answer: A synchronous flush schedule accumulates gradients for all micro-batches using one parameter version and updates only after the pipeline drains. More asynchronous schedules need explicit version management or accept different semantics.

Interviewer expects: weight version and update boundary.

Common mistake: Updating a stage after each micro-batch without discussing staleness.

Q10. How is pipeline parallelism combined with data parallelism?

Answer: Replicate the entire pipeline. Corresponding stages across replicas process different data and synchronize their parameter gradients within data-parallel groups.

Interviewer expects: orthogonal process groups.

Common mistake: All-reducing gradients between unrelated stages that own different parameters.

7. Deep-Dive Questions

Q1. Derive ideal GPipe efficiency.

For p equal stages and m micro-batches, one directional pipeline takes $m + p - 1$ stage-time slots for m useful slots per stage, so ideal utilization is $m / (m + p - 1)$ and bubble fraction is $(p - 1) / (m + p - 1)$. Training has forward and backward phases and possibly unequal costs, so this is a simplified intuition.

Q2. Why can more micro-batches reduce throughput?

They reduce bubble fraction but shrink per-micro-batch GEMMs, increase kernel and send/receive overhead, and may add recomputation or framework scheduling cost. Throughput peaks where bubble savings no longer compensate for inefficient local work.

Q3. What is the activation-memory advantage of 1F1B?

Once warmed up, each backward frees an older micro-batch's saved activations while another forward creates new ones. The maximum number of outstanding forwards on a stage is bounded by pipeline position/schedule instead of growing to all m as in all-forward/all-backward.

Q4. How do virtual stages reduce bubbles?

Each device owns multiple chunks spaced through model depth. It can switch between chunks so work returns sooner and the effective pipeline granularity increases. The trade-offs are extra transfers, more complex ordering, and possible cache/memory pressure.

Q5. What makes pipeline partitioning an optimization problem?

Layer compute, activation size, parameter memory, backward/forward ratios, device speeds, and link topology vary. Minimizing the maximum stage time under memory limits and boundary-transfer costs resembles constrained partitioning; equal parameter bytes or layers is only a heuristic.

8. Comparison Tables

Pipeline schedules

Aspect	GPipe / all-forward-all-backward	1F1B	Interleaved 1F1B
Basic pattern	All F, then all B	Alternate F/B after warmup	Alternate across multiple local chunks
Activation pressure	Higher	Lower	Schedule-dependent
Complexity	Lowest	Moderate	Highest

Aspect	GPipe / all-forward-all-backward	1F1B	Interleaved 1F1B
Bubble	Fill/drain	Fill/drain	Often smaller effective bubble
Weight semantics	Flush/synchronous	Usually flush/synchronous	Usually flush/synchronous

PP versus TP

Aspect	Pipeline parallelism	Tensor parallelism
Partition direction	Across layer depth	Within layer width/tensor dimensions
Communication	Adjacent-stage point-to-point	Frequent group collectives
Main idle cost	Bubble and imbalance	Collective latency/bandwidth
Parameter ownership	Complete layers per stage	Slices of layers
Natural placement	Can cross nodes	Prefer fastest intra-node links

9. Common Mistakes

- Saying PP splits a single matrix multiplication.
- Treating a micro-batch as an independent optimizer step.
- Ignoring the backward pipeline.
- Counting only fill/drain bubbles and ignoring stage imbalance.
- Balancing by layer count rather than measured time and memory.
- Assuming more micro-batches always improve throughput.
- Forgetting activation storage for in-flight micro-batches.
- Updating weights while older work is in flight without addressing versions.
- Ignoring skip connections or multiple tensors crossing a boundary.
- Synchronizing data-parallel gradients across the wrong process groups.

10. Edge Cases / Special Cases

- **Unequal forward/backward times:** a schedule diagram with identical slots becomes inaccurate.
- **Loss on the last stage:** targets must be available there; metrics may need return/reduction.
- **Encoder-decoder/cross-attention:** long-range dependencies complicate simple consecutive cuts.
- **Shared embeddings:** first and last stages may share weights and require synchronization.
- **Variable sequence lengths:** micro-batch compute imbalance can create transient bubbles.
- **Failure of one stage:** the whole synchronous pipeline stops.
- **Inference autoregression:** each generated token revisits all stages, so bubbles and per-token latency differ from training.
- **Small batch:** there may be too few micro-batches to fill a deep pipeline efficiently.

11. How to Explain in Interview

Pipeline parallelism assigns consecutive model layers to different stages and streams micro-batches through them. Forward activations move to the next stage and activation gradients return during backward. Micro-batching creates concurrency, but stages are idle during fill and drain, producing a bubble. Good designs balance stage time, use schedules such as 1F1B to control activation memory, and update weights only at a consistent boundary.

12. Quick Revision Notes

- **Split:** model depth into stages.
- **Traffic:** activations forward, activation gradients backward.
- **Concurrency unit:** micro-batch.
- **Main inefficiency:** fill/drain bubble plus imbalance.
- **GPipe:** all forward then all backward.
- **1F1B:** alternating steady state, fewer live activations.
- **Trap:** optimizer usually steps after the accumulated batch, not per micro-batch.

13. Practice Tasks

1. Draw a clock-slot schedule for three stages and six micro-batches.
2. Calculate ideal bubble fraction for $p=4, m=16$ and $p=8, m=16$.
3. Given per-layer timings and memory, partition 24 layers into four balanced stages.
4. Simulate GPipe and 1F1B schedules and count live activations per stage.
5. Add a slow stage to the simulation and measure throughput.
6. Identify tensors crossing candidate transformer stage boundaries.
7. Explain how corresponding stages form data-parallel groups in a hybrid setup.
8. Compare increasing micro-batch count with using activation checkpointing under a fixed memory limit.

14. Final Cheat Sheet

Item	Pipeline-parallel answer
Core definition	Split layers into stages and stream micro-batches
Why it matters	Fits deep models across devices
Forward traffic	Boundary activations
Backward traffic	Boundary activation gradients
Main schedule choice	GPipe versus 1F1B/interleaving
Main performance risk	Bubble and slowest-stage imbalance
Main correctness risk	Inconsistent weight versions or wrong stage ordering
One-line answer	"Pipeline parallelism splits model depth across stages and overlaps micro-batches like an assembly line."

Fully Sharded Data Parallel (FSDP)

1. Overview

Definition

Fully Sharded Data Parallel (FSDP) is data-parallel training in which parameters, gradients, and optimizer states are partitioned across ranks instead of permanently replicated. A rank temporarily gathers the full parameters needed for the current module, computes with them, and releases or reshapes them afterward.

Why it matters

Mixed-precision model weights are only one part of training memory. Gradients, FP32 master weights, and optimizer moments can make model state several times larger than the parameter file. FSDP divides that persistent state approximately by the data-parallel world size, allowing much larger models or batches to fit.

Where it is used

- Large language model pretraining and fine-tuning
- Large vision transformers and multimodal models
- Multi-node clusters where one model replica cannot fit on one GPU
- PyTorch systems needing ZeRO Stage 3-like sharding integrated with module execution

Why interviewers ask

FSDP tests memory accounting, collectives, transient versus persistent memory, scheduling, module granularity, mixed precision, and distributed checkpoint design.

2. Core Idea

Intuition

Imagine a four-volume encyclopedia shared by four students. Each student stores one volume permanently. When the class studies a chapter, they briefly exchange the relevant pages so everyone can use the complete chapter, then discard borrowed pages. Notes produced during study are reduced and repartitioned so each student keeps only the notes for their own volume.

Small example

A model has an 8 GB parameter set and four ranks. Ignoring padding and metadata:

At rest on each rank: about 2 GB parameter shard

Before layer group L:

shard 0 + shard 1 + shard 2 + shard 3

all-gather

|

full parameters for L briefly exist

|

```

        forward/backward computation
        |
        discard full parameters; keep shard

Backward gradients for L:
    full local contributions --reduce-scatter--> one reduced shard/rank

```

The exact peak is not simply $8 \text{ GB} / 4$: a rank may temporarily hold gathered parameters, activations, communication buffers, and prefetched next-module parameters.

Step-by-step execution

1. FSDP flattens or groups parameters and assigns shards to ranks.
2. Before a wrapped unit executes, ranks all-gather its parameter shards.
3. The unit performs forward computation using the materialized parameters.
4. Full parameters can be freed or resharded after forward.
5. During backward, parameters are gathered again if they were freed.
6. Local full gradients are reduce-scattered; each rank keeps only its reduced shard.
7. Each optimizer updates only locally owned parameter shards using locally owned optimizer-state shards.

3. Important Subtopics

What is sharded

Full sharding partitions parameters, gradients, and optimizer state. This differs from sharding only optimizer state or only gradients. The main benefit is persistent model-state memory approaching $1/N$ per rank, although nonsharded activations and temporary buffers remain.

All-gather and reduce-scatter

An **all-gather** reconstructs a full parameter group from shards before computation. A **reduce-scatter** sums gradient contributions and gives each rank only the shard it owns. Together they provide the mathematical effect of data parallelism without retaining full state.

Wrapping granularity

FSDP operates around wrapped module units. Wrapping the entire model once causes a very large gather and high peak memory. Wrapping every tiny layer creates many latency-heavy collectives. Transformer-block granularity is common because it bounds gathered memory while keeping messages large enough for bandwidth efficiency.

Reshard-after-forward policy

Freeing gathered parameters after forward saves memory but requires gathering them again in backward. Keeping them until backward uses more memory but saves communication. This is a classic memory-versus-bandwidth trade-off and can differ for root and nested units.

Prefetch and overlap

FSDP can prefetch parameters for the next forward or backward unit while the GPU computes the current unit. Aggressive prefetch improves overlap but increases peak memory because multiple gathered groups may coexist. Rate limiting can prevent the CPU from enqueueing too many all-gathers.

Mixed precision

Parameters may be gathered in BF16/FP16 for compute, gradients may be reduced in a chosen type, and optimizer state may remain FP32. These choices affect network bytes, numerical stability, and memory. Precision policy is not identical to state sharding; both must be reasoned about independently.

Original versus flattened parameters

Implementations often flatten parameters to make large contiguous communication buffers. Preserving original parameter views can improve tooling and optimizer compatibility, but it does not remove the underlying sharding. Interviewers may ask why parameter references captured before wrapping can become invalid or misleading.

Initialization of huge models

Constructing a full model on every CPU or GPU may itself exceed memory. Large systems initialize on a meta device or initialize only the owning shards, then synchronize initial values. The important requirement is that logically identical parameters start consistently without materializing unnecessary full copies.

Checkpoint formats

- **Full state dict:** easy to use elsewhere, but gathering it can need large CPU/GPU memory and rank-0 I/O.
- **Sharded state dict:** each rank writes its shard; scalable but tied to distributed checkpoint tooling.
- **Local state:** lowest-level representation and least portable.

Reshardable checkpoints are important when restarting with a different world size.

Parameters outside forward

Some models inspect or reuse parameters outside their owning module's `forward`. A parameter may only be a local shard at that moment. Such code needs an explicit full-parameter context or a compatible architecture; otherwise shape assumptions and access can fail.

4. Real-World Example

Consider fine-tuning a transformer whose training state does not fit on one 80 GB GPU. Eight GPUs wrap each transformer block independently:

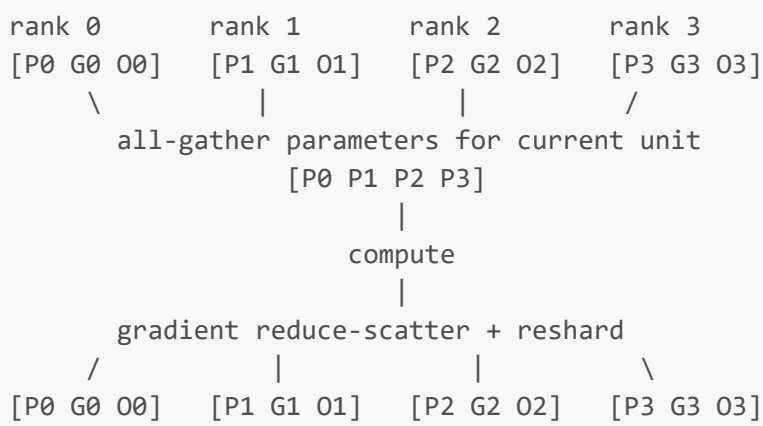
```
# Conceptual PyTorch shape; policy/details vary by version.
model = FSDP(
    model,
    auto_wrap_policy=transformer_block_policy,
    mixed_precision=bf16_policy,
)
```

```
for batch in loader:
    optimizer.zero_grad()
    loss = model(batch).loss
    loss.backward()
    optimizer.step()
```

While block 7 computes, FSDP can gather block 8. After a block no longer needs full parameters, it returns to shards. Data is still partitioned across ranks, so FSDP combines data parallelism with model-state sharding.

5. Diagrams / Mental Models

Persistent state across 4 ranks



Mental model: **shard at rest, gather just in time, reduce and reshard after use.**

Memory component	Plain DDP per rank	Fully sharded per rank, approximate
Parameters	P	P/N persistent
Gradients	G	G/N after reduction
Optimizer state	O	O/N
Activations	Local batch dependent	Local batch dependent; not automatically sharded
Peak temporary state	Small communication buckets	Gathered module parameters and buffers

6. Common Interview Questions

Q1. What problem does FSDP solve that DDP does not?

Answer: DDP replicates all model state on every GPU. FSDP shards parameters, gradients, and optimizer state, so a model whose state cannot fit on one GPU can be trained across several GPUs.

Interviewer expects: capacity versus throughput and all three state categories.

Common mistake: Saying FSDP automatically shards activations too.

Q2. How can a rank compute a layer if it stores only a parameter shard?

Answer: Before the wrapped layer runs, ranks all-gather their shards to materialize the required full parameters temporarily. The full copy is freed or resharded after its useful lifetime.

Interviewer expects: just-in-time materialization.

Common mistake: Assuming matrix multiplication operates directly on arbitrary parameter shards without communication.

Q3. Why use reduce-scatter for gradients?

Answer: It both sums data-parallel gradient contributions and distributes only the final shard owned by each rank. An all-reduce would unnecessarily leave every rank with the full gradient.

Interviewer expects: reduction plus partitioning.

Common mistake: Describing it as an ordinary scatter with no reduction.

Q4. Is FSDP memory exactly divided by world size?

Answer: Model-state storage approaches that reduction, but total and peak memory do not. Activations, temporary full parameters, prefetch buffers, allocator fragmentation, CUDA context, and unsharded modules still consume memory.

Interviewer expects: persistent versus peak memory.

Common mistake: Promising an exact N -fold reduction.

Q5. How does wrapping granularity affect performance?

Answer: Coarse wrapping gives fewer, larger collectives but higher peak gathered memory and less fine-grained overlap. Fine wrapping lowers each gathered unit's peak but creates more collective latency and scheduling overhead.

Interviewer expects: memory, latency, bandwidth, overlap trade-off.

Common mistake: Claiming the smallest possible wrapper is always best.

Q6. Why might backward gather parameters again?

Answer: Backward needs weights to compute input gradients. If full weights were resharded after forward to save memory, they must be reconstructed for backward.

Interviewer expects: parameter lifetime and reshard policy.

Common mistake: Thinking backward only needs saved activations and gradients.

Q7. What makes FSDP checkpointing harder?

Answer: No rank naturally owns the complete state. Producing a full checkpoint requires gathering, while scalable sharded checkpoints require metadata and a way to reshard when topology changes.

Interviewer expects: full versus sharded formats and world-size portability.

Common mistake: Having every rank write a full checkpoint.

Q8. Does FSDP eliminate communication?

Answer: No. It changes communication from DDP's gradient all-reduce to parameter all-gathers and gradient reduce-scatters, often scheduled per wrapped unit. It spends communication to save memory.

Interviewer expects: communication-volume/schedule trade-off.

Common mistake: Treating sharding as free memory compression.

Q9. How does mixed precision interact with FSDP?

Answer: Gathered parameters and gradient reductions can use lower precision to reduce memory and bytes, while optimizer state can remain FP32. A separate loss-scaling or BF16 strategy handles numerical range.

Interviewer expects: distinguish storage, compute, reduction, and optimizer dtypes.

Common mistake: Assuming one dtype setting applies identically to all state and communication.

Q10. When would you still choose DDP?

Answer: When the full model state fits comfortably and DDP achieves the required throughput. DDP is simpler, has fewer parameter gathers, and often performs better for smaller models.

Interviewer expects: choose sharding for a demonstrated memory need.

Common mistake: Calling FSDP universally faster because it is more advanced.

7. Deep-Dive Questions

Q1. Estimate Adam training-state memory and FSDP savings.

With mixed-precision parameters, a rough implementation might hold 2 bytes of low-precision parameter, 2 bytes of gradient, a 4-byte FP32 master parameter, and two 4-byte Adam moments: about 16 bytes per parameter before activations and temporary buffers. Full sharding across N ranks makes the persistent share roughly $16/N$ bytes per parameter per rank. Exact layouts vary, so state the assumptions.

Q2. Why can prefetching cause an out-of-memory error even when shard math says the model fits?

The current unit's full parameters, next unit's prefetched parameters, backward saved tensors, communication buffers, and allocator-reserved blocks may overlap in time. Shard math measures steady storage; OOM is determined by the temporal peak.

Q3. How do tied weights complicate FSDP?

Two modules may reference the same logical parameter. Independent flattening or wrapping can accidentally treat those references as separate ownership units or gather schedules. The wrapping policy must preserve shared identity and ensure both uses see a valid full view.

Q4. Why is module execution order important?

Prefetch and reshard scheduling predicts which parameter group will be needed next. Dynamic control flow or a different first-iteration order can reduce overlap or violate assumptions. Static transformer stacks are especially suitable because their order is predictable.

Q5. How would you choose an FSDP unit size?

Start with natural repeated blocks, measure peak memory and communication overlap, then adjust. A unit must be small enough that its gathered parameters fit alongside activations yet large enough to make collectives bandwidth-efficient. Topology and recomputation policy influence the best point.

8. Comparison Tables

FSDP versus DDP versus tensor parallelism

Aspect	DDP	FSDP	Tensor parallelism
Data	Sharded	Sharded	Usually same micro-batch within TP group
Layer parameter use	Full local replica	Full, temporarily gathered	Operation itself consumes parameter shards
Persistent state	Replicated	Sharded	Sharded along tensor dimensions
Communication timing	Mainly backward	Before units and during backward	Within layer forward and backward
Best fit	Model fits; need speed	State does not fit	Individual layers/tensors are too large

Checkpoint options

Format	Advantage	Cost/risk
Full	Portable, easy single-device load	Expensive gather and rank-0 memory/I/O
Sharded	Scalable save/load	Requires distributed metadata/tooling
Local	Closest to runtime layout	Least portable across world sizes/configurations

9. Common Mistakes

- Treating FSDP as activation sharding.
- Calculating only steady shard memory and ignoring transient all-gathers.
- Wrapping every small submodule and becoming latency-bound.
- Wrapping the entire model and then exceeding peak memory.
- Accessing a sharded parameter outside a valid full-parameter context.
- Assuming mixed precision and sharding are the same optimization.
- Saving checkpoints without deciding whether they must load at another world size.
- Creating the full model on every GPU before sharding when initialization itself is too large.
- Ignoring shared parameters, frozen parameters, or optimizer construction order.

- Expecting FSDP to outperform DDP when memory is not a constraint.

10. Edge Cases / Special Cases

- **Frozen parameters:** mixing trainable and frozen state may change gradient allocation and wrapping efficiency.
- **Shared/tied embeddings:** ownership and wrapping must preserve aliasing.
- **CPU offload:** saves GPU memory but can become PCIe- and CPU-bandwidth-bound.
- **Gradient accumulation:** suppressing synchronization across micro-batches can retain fuller gradient buffers depending on configuration; memory must be measured.
- **Hybrid sharding:** shard within a node and replicate across nodes to match faster intra-node links.
- **Tiny models:** extra gathers dominate and plain DDP is usually better.
- **Dynamic modules:** unpredictable parameter use weakens prefetching.
- **Inference/export:** a sharded training checkpoint may need consolidation or distributed loading.

11. How to Explain in Interview

FSDP is data parallelism with fully sharded model state. Each rank permanently stores only part of the parameters, gradients, and optimizer state. It all-gathers a wrapped module's parameters just before computation, then reduce-scatters gradients and returns to shards. This greatly reduces persistent memory but introduces parameter communication, transient gathered memory, and more complex wrapping and checkpointing.

12. Quick Revision Notes

- **At rest:** parameter, gradient, and optimizer shards.
- **Before compute:** parameter all-gather.
- **After backward:** gradient reduce-scatter.
- **Not automatically sharded:** activations.
- **Key tuning knob:** wrapped-unit size and prefetch/reshard policy.
- **Peak-memory trap:** overlapping gathered units and communication buffers.
- **Checkpoint trap:** runtime state is distributed, so portability needs a deliberate format.

13. Practice Tasks

1. Make a memory spreadsheet for parameters, gradients, Adam moments, and activations under DDP and 8-way FSDP.
2. Draw the gather/compute/reshard schedule for three transformer blocks.
3. Wrap a small transformer block-by-block and compare peak memory with whole-model wrapping.
4. Profile all-gather and reduce-scatter overlap on a two-GPU run.
5. Save a sharded checkpoint and reload it with a different world size if the framework supports resharding.
6. Compare reshard-after-forward enabled and disabled.
7. Explain why activation checkpointing remains useful with FSDP.
8. Identify code that accesses parameters outside **forward** and discuss how sharded views affect it.

14. Final Cheat Sheet

Item	FSDP answer
Core definition	Fully sharded parameters, gradients, and optimizer state
Why it matters	Makes otherwise-too-large training state fit
Forward collective	Parameter all-gather
Backward collective	Parameter all-gather if reshared, then gradient reduce-scatter
Main comparison	DDP replicates persistent state; FSDP shards it
Main performance choice	Wrapping granularity and overlap
Main memory trap	Temporary full parameters still exist per unit
One-line answer	"FSDP shards model state at rest and gathers only the parameters needed for the current module."

ZeRO (Zero Redundancy Optimizer)

1. Overview

Definition

ZeRO, the **Zero Redundancy Optimizer**, removes redundant data-parallel model state in stages. Stage 1 shards optimizer states, Stage 2 additionally shards gradients, and Stage 3 additionally shards parameters. Every stage preserves synchronous data-parallel training semantics while lowering per-rank memory.

Why it matters

In ordinary data parallelism, every rank stores the same weights, gradients, and optimizer states. Adam's two moments and commonly an FP32 master weight make that redundancy expensive. ZeRO lets users select how much redundancy to remove and how much communication/runtime complexity to accept.

Where it is used

- Large-scale transformer training through systems such as DeepSpeed
- Foundation-model pretraining, fine-tuning, and memory-constrained experiments
- GPU plus CPU/NVMe offload configurations
- Cluster training where model-state capacity, not only compute, limits scale

Why interviewers ask

The three stages form a clean memory-and-communication progression. Interviewers use ZeRO to test whether candidates can account for state, explain collectives, distinguish logical semantics from physical placement, and compare related systems such as FSDP.

2. Core Idea

Intuition

Four chefs use the same recipe. Plain data parallelism gives every chef a complete pantry, complete notes, and complete ingredient list. ZeRO first distributes the bulky notes (optimizer state), then distributes today's measurements (gradients), then distributes the pantry itself (parameters). Whenever everyone needs an ingredient, they exchange only what is required.

Small example

Assume model-state memory components **P** parameters, **G** gradients, and **O** optimizer state on **N** ranks:

Strategy	Approximate persistent state per rank
DDP	$P + G + O$
ZeRO-1	$P + G + O/N$
ZeRO-2	$P + G/N + O/N$
ZeRO-3	$P/N + G/N + O/N$

This table excludes activations, temporary communication buffers, fragmentation, and framework metadata.

Step-by-step progression

1. **Stage 1:** each rank owns optimizer state for a parameter partition. After gradient synchronization, it updates its partition; updated parameters are made consistent across ranks.
2. **Stage 2:** gradient contributions are reduce-scattered so each rank retains only gradients for its owned optimizer partition.
3. **Stage 3:** parameters are sharded too. Required parameter shards are all-gathered just in time for layer computation and released afterward.

3. Important Subtopics

ZeRO Stage 1: optimizer-state partitioning

Adam stores first and second moments, usually FP32, and often an FP32 master copy. Stage 1 assigns these states to ranks, so only the owner updates each parameter partition. It provides meaningful memory savings with less invasive execution than parameter sharding. Interview angle: optimizer state can dominate memory even though it is not used in forward/backward compute.

ZeRO Stage 2: gradient partitioning

Stage 2 avoids keeping a complete reduced gradient on every rank. Reduce-scatter delivers each owner the gradient partition it needs. Parameters remain replicated, so forward execution is DDP-like. It saves gradient memory while adding sharded reduction bookkeeping.

ZeRO Stage 3: parameter partitioning

Stage 3 shards parameters and gathers them before use, similar in central idea to FSDP. It offers the strongest state-memory reduction, but parameter communication lies on the forward/backward execution path and temporary full layer parameters affect the peak.

Communication volume versus live memory

Stage 1 and 2 keep full parameters, so they avoid repeated parameter all-gathers during layer execution. Stage 3 minimizes stored state but communicates parameters. The correct stage is the least aggressive one that meets the memory target at acceptable throughput.

Offload

ZeRO can move optimizer state, parameters, or both to CPU memory and, in some systems, NVMe. Capacity increases, but PCIe transfers, CPU computation, memory pinning, and storage bandwidth become part of every step. Offload is a tiered-memory algorithm, not free extra GPU memory.

Communication buckets and overlap

Large reductions/gathers are divided into buckets. Prefetching and overlapping hide latency behind compute, while persistence thresholds may keep small frequently reused parameters resident. Bucket size and prefetch distance influence both throughput and peak memory.

Memory estimation

Always state dtype and optimizer assumptions. For one billion parameters, a rough 16-byte-per-parameter Adam layout is about 16 GB of model state before activations. Eight-way full sharding suggests roughly 2 GB persistent state per rank, not a 2 GB total peak.

ZeRO-Infinity and hierarchical memory

Extensions can coordinate GPU, CPU, and NVMe tiers for models beyond aggregate GPU memory. The main principle is to fetch state before use and evict it after use. Slow tiers demand large sequential transfers and careful overlap; random small transfers ruin throughput.

ZeRO versus optimizer algorithms

ZeRO does not change Adam into a different mathematical optimizer. It changes where Adam's tensors live and how needed values are communicated. Given equivalent precision and ordering, the logical update should match unsharded data parallelism up to floating-point effects.

4. Real-World Example

A 10-billion-parameter transformer cannot store replicated mixed-precision Adam state on each GPU. On 64 GPUs, ZeRO-3 partitions all model state. During a transformer block, ranks gather the block's weights, calculate on their different data micro-batches, reduce-scatter block gradients, and retain only owned shards for the optimizer update. Activation checkpointing may separately recompute block activations, and accumulation may defer the update for several micro-batches.

Conceptual configuration:

```
{
  "zero_optimization": {
    "stage": 3,
    "overlap_comm": true,
```

```

    "offload_optimizer": { "device": "cpu" }
  }
}

```

The interview-worthy point is the policy, not the configuration syntax: full state is sharded; communication is overlapped; selected state is kept in a slower memory tier.

5. Diagrams / Mental Models

Redundancy removed one category at a time

```

DDP      : [ P full ][ G full ][ O full ] on every rank
ZeRO-1   : [ P full ][ G full ][ O shard]
ZeRO-2   : [ P full ][ G shard][ O shard]
ZeRO-3   : [ P shard][ G shard][ O shard]

```

lower memory per rank --->
more sharding machinery --->

State	Needed during	Natural Stage-3 movement
Parameters	Forward and parts of backward	All-gather before layer use
Gradients	Optimizer update	Reduce-scatter after backward production
Optimizer moments	Optimizer update only	Stay with owner; no need on every rank

6. Common Interview Questions

Q1. What redundancy does ZeRO remove?

Answer: Redundant optimizer state at Stage 1, optimizer state plus gradients at Stage 2, and all three major model-state categories including parameters at Stage 3.

Interviewer expects: exact stage progression.

Common mistake: Saying Stage 1 shards parameters.

Q2. Does ZeRO change the training algorithm?

Answer: Its goal is to preserve data-parallel optimizer semantics while changing state placement and communication. Numerical order, dtype, and implementation details can cause small differences, but it is not a new objective or optimizer rule.

Interviewer expects: logical equivalence versus physical distribution.

Common mistake: Describing ZeRO as a gradient-descent variant.

Q3. Why is optimizer-state sharding valuable?

Answer: Adaptive optimizers keep multiple FP32 tensors per parameter, often using more memory than low-precision weights and gradients combined. Those tensors are only needed by the rank updating that parameter shard.

Interviewer expects: Adam moments and optional master weights.

Common mistake: Counting only the model checkpoint size.

Q4. What collective is natural for ZeRO-2 gradients?

Answer: Reduce-scatter, because gradients must be summed across data-parallel ranks and only the owner's partition needs to remain.

Interviewer expects: combine and shard in one collective pattern.

Common mistake: All-gathering gradients.

Q5. Why does ZeRO-3 communicate parameters?

Answer: No rank stores all weights persistently, but ordinary layer computation needs the relevant complete logical weights unless tensor-parallel kernels are used. Ranks therefore gather each layer's shards before use.

Interviewer expects: sharded storage versus compute-time materialization.

Common mistake: Assuming Stage 3 is tensor parallelism.

Q6. How is ZeRO-3 different from tensor parallelism?

Answer: ZeRO-3 reconstructs a layer's logical parameters for data-parallel computation and shards them outside their useful lifetime. Tensor parallelism keeps an operation partitioned and has GPUs jointly compute different pieces of the same layer.

Interviewer expects: temporal state sharding versus operation sharding.

Common mistake: Treating every parameter partition as tensor parallelism.

Q7. What is the downside of CPU/NVMe offload?

Answer: Transfers and possibly optimizer computation move to slower tiers. If they cannot overlap with GPU compute, step time becomes limited by PCIe, CPU memory, CPU compute, or storage bandwidth.

Interviewer expects: bandwidth hierarchy and overlap.

Common mistake: Presenting offload as pure memory savings with no runtime cost.

Q8. Which ZeRO stage should you choose?

Answer: The lowest stage that satisfies peak-memory requirements at target batch size. Higher stages save more memory but add communication and integration complexity.

Interviewer expects: measurement-driven trade-off.

Common mistake: Always choosing Stage 3.

Q9. Is memory saving exactly proportional to world size?

Answer: Only the sharded categories approach $1/N$. Activations, temporary gathered tensors, replicated small tensors, communication workspaces, and fragmentation do not.

Interviewer expects: qualify the simplified formula.

Common mistake: Dividing total GPU memory by N .

Q10. How do ZeRO and activation checkpointing complement each other?

Answer: ZeRO removes redundant model state across ranks. Checkpointing reduces forward activations stored over time by recomputing them. They target different memory categories and can be combined.

Interviewer expects: model state versus activations.

Common mistake: Assuming one makes the other unnecessary.

7. Deep-Dive Questions

Q1. Why can Stage 3 have more communication than DDP?

DDP communicates each gradient, typically once per step through all-reduce. Stage 3 reduce-scatters gradients but also all-gathers parameters for forward and often again for backward after resharding. Similar byte-volume formulas do not guarantee similar time because timing, message sizes, and overlap differ.

Q2. Can ZeRO be combined with tensor and pipeline parallelism?

Yes. Process groups form different axes: tensor-parallel ranks cooperate inside layers, pipeline ranks own different layer stages, and ZeRO/data-parallel ranks shard or replicate corresponding model partitions across data replicas. Group membership must ensure each collective runs only across the intended axis.

Q3. What happens when changing world size on resume?

A checkpoint stored as old rank-local shards cannot simply map one-to-one to new ranks. A distributed checkpoint format needs global tensor metadata so shards can be read and repartitioned for the new world size. A consolidated full checkpoint is portable but expensive.

Q4. Why retain small parameters instead of repeatedly sharding them?

Tiny tensors create latency-dominated collectives and metadata overhead while saving little memory. A persistence threshold can keep them replicated. This deliberately trades a small amount of memory for fewer messages.

Q5. How does gradient accumulation interact with ZeRO-2/3?

The system must accumulate contributions without prematurely applying the optimizer. It can avoid cross-rank synchronization on intermediate micro-batches, but the ownership and buffer representation depend on the implementation. The loss must be scaled consistently, and peak memory may change if gradients remain in a less-sharded form until the synchronization boundary.

8. Comparison Tables

ZeRO stages

Feature	DDP	ZeRO-1	ZeRO-2	ZeRO-3
Parameters	Replicated	Replicated	Replicated	Sharded

Feature	DDP	ZeRO-1	ZeRO-2	ZeRO-3
Gradients	Replicated	Replicated	Sharded	Sharded
Optimizer state	Replicated	Sharded	Sharded	Sharded
Forward parameter gather	No	No	No	Yes
Relative complexity	Low	Moderate	Moderate	Highest
Typical reason	Throughput	Optimizer memory	Optimizer + gradient memory	Maximum model-state saving

ZeRO-3 versus FSDP

Aspect	ZeRO-3	FSDP
Core principle	Full model-state sharding	Full model-state sharding
Common ecosystem	DeepSpeed	PyTorch-native distributed stack
Execution unit	Parameter partitions/modules per configuration	Wrapped module units/handles
Offload ecosystem	Strong CPU/NVMe emphasis	CPU offload and distributed checkpoint support
Conceptual interview answer	More alike than different; compare implementation, policies, and tooling	

9. Common Mistakes

- Memorizing the stage numbers in the wrong order.
- Counting only weights and forgetting optimizer moments/master weights.
- Claiming ZeRO shards activations.
- Confusing Stage 3 with tensor-parallel computation.
- Assuming offload cannot reduce performance.
- Using the $\frac{1}{N}$ formula for total peak memory.
- Ignoring checkpoint resharding when world size changes.
- Choosing Stage 3 without testing whether Stage 1 or 2 already fits.
- Assuming no parameter communication occurs in Stage 3.
- Treating ZeRO and FSDP as fundamentally unrelated algorithms.

10. Edge Cases / Special Cases

- **Non-Adam optimizers:** memory ratios change with the number and dtype of state tensors.
- **Very small parameters:** replication may outperform sharding.
- **Sparse or expert models:** parameter use is conditional, complicating gather schedules and load balance.
- **Offload page faults:** unpinned or poorly scheduled CPU memory can cause unpredictable stalls.
- **NVMe endurance/capacity:** storage is not merely a bandwidth number in long runs.
- **Shared weights:** partitions must preserve aliasing and consistent updates.

- **Gradient overflow:** all ranks must agree whether a mixed-precision optimizer step is skipped.
- **Elastic resize:** checkpoint format must support resharding; live membership changes are not automatically safe.

11. How to Explain in Interview

ZeRO removes duplicated state from data-parallel training in three stages: Stage 1 shards optimizer state, Stage 2 also shards gradients, and Stage 3 also shards parameters. Stage 3 gathers parameters just before a layer and reduce-scatters gradients afterward. Higher stages save more memory but increase communication, scheduling, and checkpoint complexity.

12. Quick Revision Notes

- **Stage 1:** optimizer states sharded.
- **Stage 2:** optimizer states + gradients sharded.
- **Stage 3:** optimizer states + gradients + parameters sharded.
- **Key collectives:** reduce-scatter for gradients; all-gather for Stage-3 parameters.
- **Not covered:** activation memory.
- **Offload:** more capacity, slower memory path.
- **Trap:** a shard formula describes model state, not total peak GPU memory.

13. Practice Tasks

1. Calculate per-GPU persistent state for a 7B-parameter mixed-precision Adam model under DDP and each ZeRO stage on eight GPUs.
2. Draw Stage-3 communication for two layers and four ranks.
3. Benchmark Stage 1, 2, and 3 on the same model and record throughput and peak memory.
4. Add CPU optimizer offload and identify whether PCIe or CPU compute becomes the bottleneck.
5. Explain the difference between sharding a weight tensor for storage and sharding its matrix multiplication.
6. Design a checkpoint that can reload from eight ranks onto four ranks.
7. Combine accumulation with ZeRO and verify that loss normalization matches a large-batch baseline.
8. Decide which stage fits given a concrete state/activation memory budget.

14. Final Cheat Sheet

Item	ZeRO answer
Core definition	Progressive removal of redundant data-parallel model state
Stage sequence	Optimizer -> gradients -> parameters
Why it matters	Large memory savings with preserved data-parallel semantics
Stage-2 collective	Gradient reduce-scatter
Stage-3 addition	Just-in-time parameter all-gather
Most asked comparison	ZeRO-3 and FSDP share full-sharding principles

Item	ZeRO answer
Main trap	Offload and aggressive sharding cost bandwidth/time
One-line answer	"ZeRO stages progressively shard optimizer state, gradients, and parameters to eliminate data-parallel redundancy."

Distributed Data Parallel (DDP)

1. Overview

Definition

Distributed Data Parallel (DDP) keeps a complete model replica on every worker, gives each worker a different slice of the mini-batch, and synchronizes gradients so that every replica performs the same optimizer update. In PyTorch, a worker is normally one process bound to one GPU.

Why it matters

One GPU processes only part of the global batch, so multiple GPUs can increase samples processed per second. DDP is the usual starting point when the model and optimizer state already fit on one GPU because it changes the training algorithm very little and has predictable scaling.

Where it is used

- Multi-GPU training on one server with NVLink or PCIe
- Multi-node training over InfiniBand or Ethernet
- Computer vision, recommendation, speech, and language-model training
- Synchronous replicas in frameworks such as PyTorch, JAX, and TensorFlow

Why interviewers ask

DDP reveals whether a candidate understands processes, collectives, gradient mathematics, synchronization, data sampling, communication overlap, and the distinction between local and global batch size.

2. Core Idea

Intuition

Imagine four students each solving a different quarter of a worksheet using an identical answer key. Before updating the key, they combine what each quarter taught them. Every student then makes the same update, so all copies remain identical.

Small example

With two GPUs and a global batch of eight:

```

GPU 0: samples 0..3 -> local loss -> local gradient g0
GPU 1: samples 4..7 -> local loss -> local gradient g1
      |
      v
all-reduce SUM / 2
      |
      v
both GPUs receive (g0 + g1) / 2
      |
      v
identical optimizer step

```

If each worker computes the **mean** loss over the same number of samples and DDP averages gradients, the result equals the gradient of the mean loss over the global batch, apart from floating-point ordering.

Step-by-step execution

1. A launcher creates one process per GPU and initializes a process group.
2. Each process constructs the same model and starts with identical parameters.
3. A distributed sampler assigns disjoint training examples to ranks.
4. Each rank performs its local forward pass and computes a local loss.
5. Backpropagation produces gradients. DDP hooks place ready gradients into buckets.
6. An asynchronous all-reduce begins as soon as a bucket is ready, overlapping communication with remaining backpropagation.
7. Each rank receives the same reduced gradient.
8. Every rank independently runs the same optimizer step and stays synchronized.

For N equal-size workers with averaged local losses:

$$g = \frac{1}{N} \sum_{r=0}^{N-1} g_r$$

3. Important Subtopics

Rank, world size, and process group

- **Rank** uniquely identifies a process.
- **Local rank** usually identifies its GPU within a node.
- **World size** is the number of participating processes.
- A **process group** defines the ranks participating in collectives.

These matter because an incorrect device-to-rank mapping can make every process use GPU 0, while an incorrect world size causes initialization or collectives to hang. Interviewers often ask why one process per GPU is preferred: it isolates CUDA contexts, avoids Python-thread coordination, and maps naturally to collective libraries.

Data partitioning

A distributed sampler must give different samples to different ranks. It normally pads or drops samples so every rank executes the same number of iterations. Calling `sampler.set_epoch(epoch)` changes the

deterministic shuffle each epoch. A common interview bug is using an ordinary shuffled loader on every rank, which makes all GPUs repeat the same work.

All-reduce

All-reduce combines values across ranks and returns the result to every rank. A ring all-reduce can be understood as a reduce-scatter followed by an all-gather. Its bandwidth cost per rank is approximately:

$$2 \frac{N-1}{N} S$$

bytes for a gradient payload of size S , ignoring latency. It is bandwidth-efficient for large tensors, but many tiny collectives are latency-bound.

Gradient bucketing and overlap

DDP groups gradients into buckets. During backward, later-layer gradients become ready first. Once a bucket is complete, its all-reduce can run while earlier layers are still computing gradients. Larger buckets reduce launch latency but delay the first communication; smaller buckets start earlier but create more collectives. Interviewers look for this compute/communication trade-off.

Synchronous semantics and stragglers

Every collective waits for all participating ranks. A slow data loader, faulty GPU, network congestion, or uneven input on one rank slows everyone. DDP therefore has synchronous SGD semantics and a **straggler problem**.

Batch size and learning rate

If local batch is B , world size is N , and accumulation count is K , then:

$$B_{global} = B \times N \times K$$

Increasing world size without reducing local batch changes optimization. Linear learning-rate scaling is a heuristic, not a law; warmup and retuning may be required.

Buffers, BatchNorm, and randomness

Parameters receive gradient synchronization, but arbitrary mutable state does not. DDP can broadcast registered buffers, commonly from rank 0. Ordinary BatchNorm still computes statistics per rank; `SyncBatchNorm` communicates statistics across ranks. Random seeds should create reproducible yet nonidentical data/augmentation streams.

Uneven or unused computation

Conditional models may leave parameters unused on some iterations. DDP needs to know which gradients will not arrive; unused-parameter discovery adds graph traversal overhead. If ranks take different branches and invoke collectives in different orders, training may deadlock. Static graphs are simpler and faster.

Checkpointing

Because all ranks have identical model and optimizer state, one rank usually writes the checkpoint. A barrier may be needed around shared filesystem operations. On restart, every rank loads a consistent state; data

sampler and random-number-generator state matter for exact continuation.

4. Real-World Example

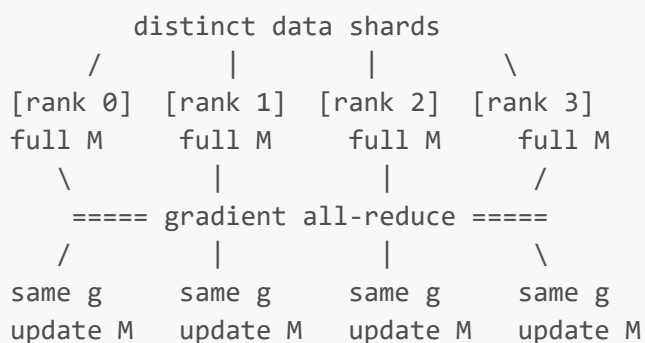
Suppose an image classifier fits on one GPU and processes 256 images per GPU. Eight GPUs use eight processes and a distributed sampler:

```
# Essential shape of a PyTorch DDP worker; initialization omitted.
torch.cuda.set_device(local_rank)
model = torch.nn.parallel.DistributedDataParallel(
    model.to(local_rank), device_ids=[local_rank]
)
sampler = torch.utils.data.DistributedSampler(dataset)
loader = DataLoader(dataset, batch_size=256, sampler=sampler)

for epoch in range(epochs):
    sampler.set_epoch(epoch)
    for x, y in loader:
        optimizer.zero_grad()
        loss = criterion(model(x.to(local_rank)), y.to(local_rank))
        loss.backward()      # gradient all-reduces are triggered here
        optimizer.step()
```

The nominal global batch is $256 \times 8 = 2048$. The model is still fully stored on every GPU, so DDP improves throughput but does not make an oversized model fit.

5. Diagrams / Mental Models



Mental model: **replicate model, partition data, average gradients.**

Scaling bottleneck	Symptom	Typical response
Compute-bound	Near-linear scaling	Add GPUs until communication dominates
Bandwidth-bound	Large buckets take long	Faster interconnect, compression where valid, more compute per step
Latency-bound	Many tiny collectives	Bucket gradients, fuse small parameters

Scaling bottleneck	Symptom	Typical response
Input-bound	GPUs wait before forward	Parallel loading, pinned memory, preprocessing improvements
Straggler-bound	Collective waits on one rank	Profile per-rank timing and balance inputs

6. Common Interview Questions

Q1. How does DDP differ from single-process data parallelism?

Answer: DDP normally runs one process per GPU and synchronizes gradients through collectives. Older single-process approaches copy the model from a primary device, scatter inputs, gather outputs, and often bottleneck on one process/device. DDP has better isolation and overlap.

Interviewer expects: process-per-GPU, replicated models, all-reduce, reduced primary-GPU bottleneck.

Common mistake: Saying DDP splits the model across GPUs; it normally replicates it.

Q2. Why does gradient averaging reproduce a larger batch?

Answer: The gradient operator is linear over a sum. If every rank uses a mean loss over an equal local batch, averaging local gradients equals differentiating the mean loss over their union.

Interviewer expects: equal weighting and reduction semantics.

Common mistake: Ignoring unequal local batch sizes; averaging rank means then weights a small rank as much as a large rank.

Q3. What is all-reduce?

Answer: A collective that reduces tensors from all ranks with an operation such as sum and distributes the result back to every rank. DDP uses it so all replicas receive matching gradients.

Interviewer expects: both reduction and redistribution.

Common mistake: Describing reduce-to-rank-0, which does not return the result everywhere.

Q4. Why can DDP hang?

Answer: Collectives must be invoked by all group members in a compatible order and with compatible tensor shapes. A rank crash, skipped backward pass, different control-flow branch, mismatched iteration count, or wrong process-group configuration can leave peers waiting.

Interviewer expects: collective ordering and rank participation.

Common mistake: Treating every hang as a slow network.

Q5. Does DDP reduce model memory?

Answer: No. Each rank stores full parameters, gradients, optimizer state, and its local activations. Local activation memory may fall when the global batch is divided, but model-state memory is replicated.

Interviewer expects: throughput versus capacity distinction.

Common mistake: Claiming N GPUs allow an N -times larger model under plain DDP.

Q6. How is communication overlapped with computation?

Answer: Autograd hooks mark gradients ready. DDP packs them into buckets and launches an asynchronous reduction when all gradients in a bucket are ready while backpropagation continues for other buckets.

Interviewer expects: readiness order, buckets, asynchronous collectives.

Common mistake: Saying one all-reduce starts only after all backward computation ends.

Q7. What happens to BatchNorm in DDP?

Answer: Standard BatchNorm computes statistics from each rank's local batch, so ranks can use different batch statistics. Synchronized BatchNorm communicates statistics to approximate a global batch statistic; evaluation uses stored running buffers.

Interviewer expects: parameters are synchronized but batch statistics need special treatment.

Common mistake: Assuming gradient all-reduce automatically combines forward-pass statistics.

Q8. How should the dataset be handled?

Answer: Use rank-aware sharding so ranks process disjoint samples, keep per-rank iteration counts compatible, and reseed deterministic shuffling per epoch. Evaluation outputs and metrics must also be reduced correctly.

Interviewer expects: distributed sampler and `set_epoch`-style reshuffling.

Common mistake: Letting every rank read the same shuffled sequence.

Q9. What determines DDP scaling efficiency?

Answer: The compute-to-communication ratio, interconnect bandwidth and latency, model gradient size, bucket schedule, input pipeline, load balance, topology, and global-batch optimization effects.

Interviewer expects: both system throughput and statistical efficiency.

Common mistake: Assuming twice the GPUs always halves wall-clock training time.

Q10. Why does only rank 0 usually save a checkpoint?

Answer: Replicated states are normally identical, so multiple ranks writing the same path is redundant and unsafe. Rank 0 writes once, with synchronization as needed.

Interviewer expects: avoid races and duplicated I/O.

Common mistake: Forgetting that rank-local RNG, sampler, or scaler state may also be needed for exact restart.

7. Deep-Dive Questions

Q1. How does a ring all-reduce work?

The tensor is divided into N chunks. During reduce-scatter, ranks circulate chunks around a ring and accumulate contributions until each rank owns one fully reduced chunk. During all-gather, those chunks circulate again so every rank reconstructs the result. It uses bandwidth efficiently but takes multiple communication steps, making small messages latency-sensitive.

Q2. When is DDP's simple average mathematically wrong?

It is wrong when local losses represent different numbers or weights of examples but rank gradients are averaged equally. The correct global mean weights each local gradient by its valid sample count. Padding duplicates, masked tokens, variable sequence lengths, and token-level losses can all require explicit numerator/denominator reductions.

Q3. Why can parameter registration order affect performance?

Buckets are formed from parameters, while backward makes gradients ready roughly in reverse execution order. A poor ordering can delay completion of a bucket even when most of its gradients are ready, reducing overlap. Frameworks may rebuild buckets after observing the first backward order.

Q4. How would you debug a distributed deadlock?

First identify the last collective reached by each rank using distributed debug logs and per-rank traces. Check equal iteration counts, conditional branches, unused parameters, exceptions before collective calls, tensor shape agreement, rendezvous configuration, and network reachability. Add monitored barriers only to localize the divergence, not as a performance fix.

Q5. What is a DDP communication hook?

A communication hook intercepts bucket reduction and can implement algorithms such as gradient compression or custom collectives. It changes numerical and convergence behavior, so it needs error-feedback or accuracy validation; reducing bytes is useful only if encode/decode cost and lost overlap do not dominate.

8. Comparison Tables

DDP versus FSDP

Aspect	DDP	FSDP
Parameters at rest	Full replica per rank	Sharded per rank
Gradients	Full after all-reduce	Sharded after reduce-scatter
Optimizer state	Full replica	Sharded
Main goal	Throughput	Memory capacity and throughput
Main communication	Gradient all-reduce	Parameter all-gather + gradient reduce-scatter
Simplicity	Usually simpler	More wrapping, scheduling, and checkpoint complexity

Collective operations

Collective	Input on each rank	Output	Common use
Broadcast	Source has tensor	Same tensor on all ranks	Initial state/buffers
Reduce	Every rank contributes	Reduced result on one rank	Aggregated logging
All-reduce	Every rank contributes	Reduced result on all ranks	DDP gradients
Reduce-scatter	Every rank contributes	Different reduced shard per rank	Sharded gradients
All-gather	Different shard per rank	All shards on all ranks	Reconstruct parameters

9. Common Mistakes

- Assuming DDP partitions model parameters.
- Forgetting to shard the input dataset.
- Confusing local batch size with global batch size.
- Changing world size without reconsidering the learning rate and schedule.
- Reducing an already averaged metric without weighting by sample count.
- Calling a collective on only one rank or in a different order.
- Performing validation on every rank and reporting only rank 0's local metric.
- Saving simultaneously to one checkpoint path from every process.
- Assuming equal seeds should produce identical augmentation on all ranks.
- Benchmarking without warmup or without synchronizing asynchronous GPU work.

10. Edge Cases / Special Cases

- **Uneven final batches:** ranks must still execute compatible step counts; dropping, padding, or a join mechanism may be necessary.
- **Variable-length token batches:** normalize by the total number of valid tokens, not blindly by ranks.
- **Gradient clipping:** clip the synchronized global gradient, normally after backward and after mixed-precision unscaling.
- **No-gradient regions:** evaluation needs no backward collectives, but metrics still require global aggregation.
- **Multiple models:** collective order must match across ranks even when models are conditionally used.
- **Sparse gradients:** not every backend/collective supports them with the same path as dense gradients.
- **Fault tolerance:** classic synchronous DDP does not transparently survive a lost rank; elastic launch can restart membership but application state must be restored.
- **Floating-point reproducibility:** different world sizes and reduction trees change summation order.

11. How to Explain in Interview

DDP is synchronous data parallelism: every GPU holds the full model but trains on a different data shard. During backward, gradients are bucketed and all-reduced, usually overlapped with remaining backpropagation. Because every rank receives the same averaged gradients and applies the same optimizer step, replicas stay synchronized. It is excellent for throughput when the model fits on one GPU, but it does not shard model state.

12. Quick Revision Notes

- **Definition:** full model replica per rank, distinct data, synchronized gradients.
- **Core collective:** all-reduce.
- **Global batch:** local batch x world size x accumulation steps.
- **Performance:** maximize compute/communication overlap and avoid stragglers.
- **Correctness:** same collective order, disjoint data shards, correctly weighted loss/metrics.
- **DDP vs FSDP:** replication versus state sharding.
- **Trap:** gradient synchronization does not automatically synchronize arbitrary forward state.

13. Practice Tasks

1. Launch a two-GPU DDP script and print rank, local rank, world size, and sampled indices.
2. Verify numerically that a one-GPU batch of eight and two equal DDP batches of four produce matching gradients.
3. Deliberately remove the distributed sampler and observe duplicated sample indices.
4. Add per-rank artificial delay and measure how it affects step time.
5. Profile a backward pass and locate overlap between gradient collectives and kernels.
6. Compute a global validation mean correctly from per-rank loss sums and sample counts.
7. Draw ring all-reduce for four ranks and four tensor chunks.
8. Explain why unequal valid-token counts require weighted gradient normalization.

14. Final Cheat Sheet

Item	DDP answer
Core definition	Replicated model + sharded data + synchronized gradients
Why it matters	Scales training throughput across GPUs
Model memory saving	None for parameters/gradients/optimizer state
Most asked mechanism	Bucketed gradient all-reduce
Most asked comparison	DDP replicates state; FSDP/ZeRO shard it
Main performance risk	Communication, input stalls, or stragglers
Main correctness risk	Mismatched collectives or duplicated/uneven data
One-line answer	"DDP gives every GPU different data, averages their gradients, and keeps full model replicas in sync."

Activation Checkpointing

1. Overview

Definition

Activation checkpointing, also called **gradient checkpointing**, reduces training memory by saving only selected forward-pass tensors and recomputing missing intermediate activations during backward. It trades extra computation for a smaller activation-memory footprint.

Why it matters

Backpropagation needs intermediate values from forward. Deep models, long sequences, and large micro-batches can make these saved activations larger than model state. Checkpointing can make a run fit without changing the model's mathematical function or permanently moving state to another device.

Where it is used

- Transformer training with long contexts
- Deep residual networks and vision transformers
- FSDP/ZeRO training where state is already sharded but activations dominate
- Pipeline stages that retain activations for several in-flight micro-batches

Why interviewers ask

The technique tests whether candidates understand autograd's saved tensors, liveness, compute-memory trade-offs, recomputation correctness, random state, side effects, and why lower peak memory can sometimes improve overall throughput.

2. Core Idea

Intuition

When solving a long calculation, you can keep every intermediate result on paper or keep only a few milestones and redo the arithmetic between them when needed. The second approach uses less paper but more time.

Small example

For four functions:

$$x_1 = f_1(x_0), \quad x_2 = f_2(x_1), \quad x_3 = f_3(x_2), \quad x_4 = f_4(x_3)$$

Normal autograd saves **x1**, **x2**, and **x3** for backward. If **f2..f4** are one checkpointed region, it may save only the region input **x1**. During backward it reruns **f2**, **f3**, and **f4** to recreate **x2** and **x3**, then computes gradients.

Normal:

```
forward  x0 -> [x1 saved] -> [x2 saved] -> [x3 saved] -> x4
backward      uses x1           uses x2           uses x3
```

Checkpointed region:

```
forward  x0 -> [x1 saved] -> x2 -> x3 -> x4
backward      rerun from x1 to regenerate x2/x3, then differentiate
```

Step-by-step execution

1. Choose a function or block as a checkpointed region.
2. Run its forward computation while retaining only boundary inputs and explicitly required tensors rather than all internal saved tensors.
3. Continue the rest of forward normally.
4. When backward reaches the region, restore needed RNG/autocast context.
5. Rerun the region's forward to rebuild internal activations.
6. Immediately consume those activations to calculate gradients and free them.

3. Important Subtopics

What autograd normally saves

Backward formulas may need layer inputs, outputs, masks, indices, or statistics. Autograd records a graph and saves selected tensors, not necessarily every named activation. Checkpointing changes which tensors remain live; it does not eliminate the computation graph or gradient storage.

Checkpoint boundaries

A boundary activation must remain available so recomputation can start. More segments mean more boundaries but shorter recomputation regions and potentially lower local peaks. Poor boundaries can leave a large tensor saved outside the region, delivering little benefit.

Reentrant and non-reentrant implementations

Frameworks may provide different checkpoint engines. A reentrant approach reruns a function through a nested backward mechanism and can impose restrictions on detached tensors or nested structures. A non-reentrant approach records more graph information and can stop recomputation after required tensors are rebuilt. Exact APIs vary; interview answers should focus on semantics and mention implementation constraints.

RNG and stochastic layers

Dropout must use the same random mask during recomputation as in the original forward, or the backward corresponds to a different computation. Frameworks commonly save and restore CPU/GPU RNG state around the region. This preservation has overhead and becomes subtle if tensors move to unexpected devices.

Stateful and side-effecting operations

The checkpointed forward runs twice. Mutating a counter, updating an external cache, consuming a queue, doing I/O, or changing global state can happen twice. BatchNorm running-stat updates are a classic concern. A checkpointed region should behave like a deterministic function of its inputs and preserved state.

Selective checkpointing

Not all operations have the same activation bytes or recompute cost. Matrix multiplies may be expensive to rerun, while elementwise operations may be cheap. Selective policies save costly outputs and recompute cheap, memory-heavy chains, seeking a better point than checkpointing entire blocks uniformly.

Memory complexity intuition

For a simple chain of L equal layers, storing every layer uses $O(L)$ activation memory. With appropriately spaced checkpoints and recomputation, an idealized schedule can approach $O(\sqrt{L})$ saved checkpoints for $O(L)$ extra work. Real transformer-block checkpointing usually reports measured bytes rather than relying on this simplified bound.

Interaction with mixed precision

Recomputation must use compatible autocast settings and dtypes. If it runs under a different precision mode, it may produce numerically different values or invalid gradients. Lower-precision activations already use fewer bytes, but checkpointing can still save substantial memory.

Interaction with distributed training

Checkpointing is mostly local compute and does not inherently shard state. With FSDP/ZeRO-3, recomputation can trigger parameter materialization again depending on reshard policy. With pipeline parallelism, it lowers per-micro-batch saved activations but adds recompute to stage time and can change pipeline balance.

Checkpointing versus saved-tensor offload

Checkpointing discards and recomputes activations. Offload keeps them in CPU memory or another tier and transfers them back. The choice compares GPU compute with transfer bandwidth/latency and available host memory.

4. Real-World Example

A 32-block transformer runs out of memory at a long sequence length. Checkpointing each transformer block retains block inputs but discards internal attention scores, projection intermediates, and MLP activations that autograd would otherwise save. During backward, each block forward is rerun once.

```
from torch.utils.checkpoint import checkpoint

for block in model.blocks:
    hidden = checkpoint(block, hidden, use_reentrant=False)
```

The actual wrapper must pass masks and other inputs correctly and keep the block deterministic during recomputation. Peak memory falls; step time rises due to repeated forward kernels. If the saved memory permits a larger micro-batch, fewer accumulation steps may partly offset that cost.

5. Diagrams / Mental Models

```
Memory over a normal forward:
layer 1: A1
layer 2: A1 A2
layer 3: A1 A2 A3
layer 4: A1 A2 A3 A4    <- high activation peak

With two checkpoint regions:
```

```

layer 1: C1
layer 2: C1
layer 3: C1 C2
layer 4: C1 C2      <- boundaries retained
backward: recreate one region, use it, free it

```

Mental model: **store milestones, replay the route during backward.**

Strategy	GPU activation memory	Extra work	Main risk
Save all	Highest	None	OOM
Checkpoint	Lower	Recompute forward operations	RNG/state inconsistency
CPU offload	Lower	Transfers	PCIe/host-memory bottleneck
Smaller micro-batch	Lower	More accumulation/smaller kernels	Lower utilization

6. Common Interview Questions

Q1. What is activation checkpointing?

Answer: It saves only selected forward boundaries and recomputes discarded intermediate activations during backward, exchanging compute time for memory.

Interviewer expects: recomputation during backward, not disk checkpoint files.

Common mistake: Confusing it with saving model weights for fault recovery.

Q2. Why are activations needed in backward?

Answer: Many derivative formulas depend on forward inputs or outputs—for example, a weight gradient uses layer input, and nonlinear derivatives depend on the activation or preactivation. Autograd saves these tensors unless it can reconstruct them.

Interviewer expects: concrete derivative dependency.

Common mistake: Saying backward needs only the scalar loss.

Q3. Does checkpointing reduce parameter or optimizer memory?

Answer: No. It primarily reduces saved forward activations. Parameters, gradients, and optimizer states need separate techniques such as FSDP/ZeRO or lower precision.

Interviewer expects: memory-category distinction.

Common mistake: Applying a generic “memory divided” claim to all tensors.

Q4. How much computation is added?

Answer: It depends on region selection. Checkpointing each block typically repeats the checkpointed forward work during backward, while the original backward still runs. It does not normally repeat the entire training step or optimizer update.

Interviewer expects: extra forward work, measured by selected region.

Common mistake: Saying compute exactly doubles in all cases.

Q5. Why is dropout tricky?

Answer: Recomputed forward must reproduce the original dropout mask. Frameworks preserve RNG state or use a controlled stateless RNG; otherwise gradients describe a different stochastic graph.

Interviewer expects: deterministic replay.

Common mistake: Disabling dropout as the only solution.

Q6. Can checkpointing change model accuracy?

Answer: Correct deterministic replay should yield equivalent gradients up to floating-point effects. Different RNG, autocast context, mutable state, or nondeterministic kernels can cause divergence.

Interviewer expects: correctness conditions.

Common mistake: Claiming recomputation is always bit-identical.

Q7. Where should checkpoints be placed?

Answer: Around memory-heavy regions with acceptable recompute cost, often repeated transformer blocks. Boundaries should balance peak live tensors; placement should be selected using memory and runtime profiling.

Interviewer expects: liveness and cost, not arbitrary every- k placement.

Common mistake: Checkpointing tiny cheap-memory wrappers while large outputs remain saved outside.

Q8. Why can checkpointing sometimes improve throughput despite recompute?

Answer: The saved memory may permit a larger micro-batch, improve GEMM efficiency, reduce accumulation steps, prevent paging/offload, or make a feasible configuration possible. Raw per-step time increases, but samples per second may not fall proportionally.

Interviewer expects: distinguish step latency from end-to-end throughput.

Common mistake: Saying extra compute can never help performance.

Q9. How does it interact with FSDP?

Answer: They target different memory: checkpointing reduces activations, FSDP shards model state. If FSDP reshards parameters after forward, recomputation may require additional all-gathers, so schedules must be profiled together.

Interviewer expects: composability plus communication side effect.

Common mistake: Assuming independent costs simply add with no timing interaction.

Q10. What operations should not be naively checkpointed?

Answer: Operations with externally visible side effects, nonreproducible randomness, input mutation, device-dependent state changes, or forward behavior that differs during recomputation.

Interviewer expects: pure/replayable region.

Common mistake: Mentioning only computational expense and ignoring correctness.

7. Deep-Dive Questions

Q1. How would you reason about the optimal segmentation of a linear chain?

Balance boundary storage against the largest region's transient recomputation memory and cost. Equal-sized layers give a square-root-style checkpoint spacing intuition, but real layers have unequal tensor sizes and runtimes, so formulate the problem using measured saved bytes and recompute time.

Q2. What happens if a checkpointed function detaches a tensor?

Detaching changes gradient connectivity. Some checkpoint implementations reject this or produce outputs that do not require gradients as expected; reentrant variants have particular restrictions. The region must preserve the intended autograd graph, and detached branches should be handled explicitly.

Q3. Why can recomputation alter a distributed communication schedule?

If checkpointed layers also use tensor-parallel collectives or FSDP parameter gathers, rerunning their forward repeats those communications. Extra operations may sit on the critical path or interact with prefetching, so checkpointing is not always purely local in a sharded model.

Q4. What is selective activation checkpointing?

It applies a policy at operation level: save outputs of expensive operations and recompute cheaper ones, subject to memory goals. This can dominate uniform block checkpointing but needs operator cost/size knowledge and runtime support.

Q5. How would you verify correctness?

Run a small deterministic model with and without checkpointing from identical parameters, inputs, RNG state, precision, and optimizer state. Compare loss and gradients within an appropriate tolerance, then test stochastic layers and stateful modules specifically. Also profile peak allocated memory and step time to confirm the intended trade-off.

8. Comparison Tables

Activation checkpointing versus related methods

Method	Reduces	Pays with	Mathematical role
Activation checkpointing	Saved activations	Recompute	Same batch/update semantics
Gradient accumulation	Per-micro-batch activation peak / batch constraint	More serial micro-batches	Builds a larger effective batch

Method	Reduces	Pays with	Mathematical role
FSDP/ZeRO	Redundant model state	Communication/complexity	Distributed state placement
Activation offload	GPU-resident activations	Host transfer and host memory	Same saved values, different tier

Checkpoint granularity

Granularity	Memory saving	Recompute/scheduling traits
Whole large region	Few boundaries, potentially large saving	Large replay and transient peak
Per block	Predictable, common compromise	Roughly one replay per block
Selective operators	Best cost-aware potential	Needs detailed policy/tooling
None	No recompute	Highest saved-activation memory

9. Common Mistakes

- Confusing activation checkpoints with training restart checkpoints.
- Saying checkpointing makes forward cheaper.
- Assuming it reduces optimizer state.
- Forgetting that checkpointed forward executes twice.
- Ignoring dropout/RNG restoration.
- Placing mutations or I/O inside a checkpointed region.
- Comparing only allocated memory after forward rather than peak over the whole step.
- Checkpointing everything without profiling recompute cost.
- Assuming block inputs are free; boundaries still consume memory.
- Forgetting repeated distributed communication during recomputation.

10. Edge Cases / Special Cases

- **No input/output requires gradients:** some implementations cannot establish the needed backward path.
- **Multiple devices inside a region:** RNG restoration may not cover devices introduced dynamically.
- **Autocast changes:** backward-time recompute must restore forward precision context.
- **In-place operations:** recomputation and version counters can expose mutation errors.
- **BatchNorm/state updates:** running statistics may update twice.
- **Nondeterministic kernels:** values may differ despite matching RNG state.
- **Nested checkpointing:** valid in some engines but can amplify overhead and complicate graphs.
- **Long sequences:** attention activations may grow quadratically unless a memory-efficient attention kernel already avoids storing them.

11. How to Explain in Interview

Activation checkpointing saves memory by retaining only selected forward boundaries. During backward, it reruns each checkpointed region to regenerate the internal activations needed for

derivatives, then frees them. It mainly targets activation memory and costs extra forward computation. Correctness requires deterministic replay, especially for dropout, autocast, and stateful operations.

12. Quick Revision Notes

- **Definition:** discard internal forward activations; recompute them in backward.
- **Saves:** activation memory, not parameters/optimizer state.
- **Costs:** extra forward compute and possibly repeated distributed communication.
- **Best boundary:** memory-heavy, replayable, reasonably cheap region.
- **Correctness trap:** RNG and mutable state.
- **Common pairing:** FSDP/ZeRO plus checkpointing.
- **Name trap:** not a fault-tolerance checkpoint.

13. Practice Tasks

1. Measure peak memory and step time for a model with and without per-block checkpointing.
2. Add dropout and verify gradients match when RNG state is preserved.
3. Put a counter side effect inside a checkpointed function and observe it execute twice.
4. Draw activation lifetimes for six layers under no checkpointing and two checkpoint regions.
5. Profile checkpointing alongside FSDP and count repeated parameter gathers.
6. Compare checkpointing with halving micro-batch size plus accumulation.
7. Identify which attention/MLP tensors dominate saved memory using a profiler.
8. Design a selective policy that saves matrix-multiply outputs but recomputes cheap elementwise operations.

14. Final Cheat Sheet

Item	Activation-checkpointing answer
Core definition	Save boundaries and recompute internal activations in backward
Why it matters	Reduces activation peak for deep/long models
Memory saved	Forward saved tensors
Cost	Extra forward computation
Most asked correctness issue	Reproduce RNG/autocast/state
Main comparison	FSDP shards state; checkpointing recomputes activations
Main trap	"Checkpoint" here does not mean saving a restart file
One-line answer	"Activation checkpointing trades extra forward recomputation for lower saved-activation memory."

Gradient Accumulation

1. Overview

Definition

Gradient accumulation computes gradients over several smaller **micro-batches**, adds those gradients into the same parameter-gradient buffers, and performs one optimizer update after all accumulation steps. It emulates a larger effective batch when that batch cannot be processed at once.

Why it matters

Activation memory usually scales with local micro-batch size. Accumulation lets training use a small memory-fitting micro-batch while preserving a larger batch for optimization. In distributed training, it can also avoid gradient synchronization on intermediate micro-batches and communicate once at the update boundary.

Where it is used

- Large language-model training and fine-tuning
- Long-sequence training with limited activation memory
- Stable large-batch optimization on small GPU counts
- Pipeline parallelism, where the batch is naturally divided into micro-batches
- Distributed training that wants more compute per gradient synchronization

Why interviewers ask

Accumulation looks simple but exposes common errors in loss normalization, zeroing gradients, optimizer/scheduler cadence, distributed synchronization, mixed-precision overflow, BatchNorm behavior, and the distinction between mathematical and systems equivalence.

2. Core Idea

Intuition

Suppose a measuring cup holds only one quarter of a recipe. Measure four quarters into a bowl, then mix once. The cup-sized portions are micro-batches; the bowl is the gradient buffer; mixing once is the optimizer step.

Small example

To emulate batch size eight using micro-batches of two, accumulate four times:

```
optimizer.zero_grad()
for i, (x, y) in enumerate(loader):
    loss = criterion(model(x), y) / 4
    loss.backward()           # adds into existing .grad buffers

    if (i + 1) % 4 == 0:
        optimizer.step()
        optimizer.zero_grad()
```

For equal micro-batches and mean-reduced loss:

$$g_{effective} = \frac{1}{K} \sum_{k=1}^K g_k$$

Dividing each loss by K makes accumulated gradients equal the gradient of the mean loss over the combined batch, aside from floating-point order and stateful-layer behavior.

Step-by-step execution

1. Clear gradients once at the start of an accumulation window.
2. Run forward on micro-batch 1 and compute a correctly normalized loss.
3. Run backward; gradients are added to parameter `.grad` buffers.
4. Repeat forward/backward for the remaining micro-batches without clearing gradients.
5. If using DDP, skip cross-rank synchronization for intermediate micro-batches.
6. Unscale mixed-precision gradients if necessary, then clip/check them once.
7. Perform one optimizer step and one update-cadence scheduler step.
8. Clear gradients and begin the next window.

3. Important Subtopics

Effective global batch size

For local micro-batch B , N data-parallel ranks, and K accumulation steps:

$$B_{effective} = B \times N \times K$$

For variable-length data, examples may not be the right unit; effective tokens can be the sum of valid tokens across micro-batches and ranks.

Gradient addition semantics

Autograd adds into existing gradient buffers by default. `optimizer.zero_grad()` or setting gradients to `None` clears the previous window. Clearing after every micro-batch destroys accumulation; never clearing causes gradients to leak across optimizer steps.

Loss scaling and normalization

If every micro-batch has equal size and its loss is a mean, divide by K . If sizes or valid-token counts differ, sum loss numerators and normalize by the total valid count. Simply averaging micro-batch means weights every micro-batch equally, which is wrong when denominators differ.

Optimizer cadence

Momentum, Adam moments, weight decay, parameter update count, and bias correction advance on `optimizer.step()`, not on each backward. Accumulation therefore matches a large batch better than taking a small optimizer step after each micro-batch; those are different optimization trajectories.

Learning-rate scheduler cadence

A scheduler defined per optimizer update should step once per accumulation window. A scheduler defined per epoch or metric follows its own contract. Accidentally stepping per micro-batch compresses the schedule

by factor **K**.

Distributed **no_sync**

DDP normally all-reduces gradients on every backward. During accumulation, a no-synchronization context can skip intermediate all-reduces and synchronize only the last backward:

```
for micro in range(K):
    context = model.no_sync() if micro < K - 1 else nullcontext()
    with context:
        (model(batch[micro]).loss / K).backward()
```

This reduces network operations. The forward must typically occur inside the context as required by the wrapper's hook setup.

Mixed precision and loss scaling

Scale every micro-batch loss and accumulate scaled gradients. At the update boundary, unscale once, detect overflow consistently, clip once, call the scaler/optimizer step, and update the scale. An overflow policy may discard the entire accumulation window; distributed ranks must agree on skipping the update.

Gradient clipping

Clipping each micro-batch separately changes the direction and magnitude compared with clipping the final combined gradient. For large-batch equivalence, accumulate, unscale, then clip once immediately before the optimizer step.

Batch-dependent layers

Accumulation combines gradients but does not make forward computations see the combined batch. BatchNorm statistics are computed per micro-batch, contrastive losses see fewer negatives, and other cross-example operations differ. Thus accumulation is not always mathematically equivalent to a physically large batch.

Throughput and memory

Smaller micro-batches lower activation peak but may underutilize the GPU. More accumulation steps also repeat launch and input overhead. Conversely, avoiding intermediate all-reduces increases compute per synchronization. Select **B** as large as fits efficiently, then choose **K** for the target effective batch.

Remainder windows

If an epoch ends before **K** micro-batches, either perform a smaller final update with correct normalization or carry/skip it deliberately. Dividing by fixed **K** underweights a partial final window. Distributed ranks must reach update boundaries consistently.

4. Real-World Example

A fine-tuning job uses four GPUs. Each GPU fits two sequences, and the desired effective batch is 64 sequences:

```
local micro-batch B = 2
world size N       = 4
accumulation K      = 8
effective batch     = 2 x 4 x 8 = 64
```

Each rank performs eight forwards/backwards on different local micro-batches. The first seven skip DDP gradient all-reduce. The eighth synchronizes the accumulated gradient; then every rank clips, updates, advances the scheduler, and clears gradients. If sequences contain different valid-token counts, the job normalizes by the window's global valid-token total rather than assuming all eight micro-batches are equal.

5. Diagrams / Mental Models

one optimizer update

```
micro 1: forward -> backward -> grad buffer += g1 -- no sync
micro 2: forward -> backward -> grad buffer += g2 -- no sync
micro 3: forward -> backward -> grad buffer += g3 -- no sync
micro 4: forward -> backward -> grad buffer += g4 -- synchronize
                                     |
                               unscale / clip
                                     |
                               optimizer.step
                                     |
                               zero gradients
```

Mental model: **many backward additions, one optimizer update.**

Quantity	Advances every micro-batch?	Advances every accumulation window?
Forward/backward	Yes	Contains K executions
Parameter .grad content	Accumulates	Cleared after update
Optimizer moments	No	Yes
Parameter values	No	Yes
Update-based LR scheduler	No	Yes
DDP all-reduce with no_sync	Only final micro-batch	Once

6. Common Interview Questions

Q1. What is gradient accumulation?

Answer: It sums gradient contributions from multiple memory-fitting micro-batches before one optimizer update, creating a larger effective batch.

Interviewer expects: multiple backwards, one step.

Common mistake: Calling multiple optimizer steps “accumulation.”

Q2. How do you calculate effective batch size?

Answer: Local micro-batch times data-parallel world size times accumulation steps, assuming each rank has equal valid sample counts.

Interviewer expects: $B \times N \times K$.

Common mistake: Forgetting the data-parallel factor or confusing pipeline stage count with data replicas.

Q3. Why divide loss by accumulation steps?

Answer: Backward adds gradients. Dividing equal-size micro-batch mean losses by K makes their sum equal the mean gradient of the combined batch rather than a gradient K times larger.

Interviewer expects: sum versus mean convention.

Common mistake: Dividing again after the optimizer has already consumed gradients.

Q4. When is division by K incorrect?

Answer: When micro-batches have unequal numbers of valid samples/tokens or losses use a different reduction. Then accumulate summed loss contributions and divide by the true total denominator, or weight each micro-batch accordingly.

Interviewer expects: unequal-size weighting.

Common mistake: Averaging averages without considering their counts.

Q5. Is accumulation exactly equivalent to one large batch?

Answer: It is equivalent for additive per-example losses with consistent parameters and correct normalization, up to floating-point order. It differs for BatchNorm, cross-example losses, stochastic/stateful behavior, or if parameters update between micro-batches.

Interviewer expects: conditions for equivalence.

Common mistake: Giving an unconditional yes.

Q6. Where should `zero_grad()` be called?

Answer: Before the first micro-batch of a window or immediately after the previous optimizer step, not between micro-batches. Clear again after the update before the next window.

Interviewer expects: gradient buffers persist exactly for one window.

Common mistake: Clearing inside every micro-batch loop.

Q7. How do you avoid redundant DDP communication?

Answer: Disable synchronization for intermediate micro-batches and allow the final backward to trigger reduction of the accumulated gradients.

Interviewer expects: a `no_sync`-style mechanism and final synchronization.

Common mistake: Disabling synchronization on every micro-batch, leaving replicas inconsistent.

Q8. When should gradient clipping occur?

Answer: Once after all micro-batches have accumulated and after mixed-precision unscaling, immediately before the optimizer step.

Interviewer expects: clip the effective gradient.

Common mistake: Clipping every micro-gradient independently.

Q9. How should a learning-rate scheduler be stepped?

Answer: If it is update-based, step it once per optimizer update, not once per micro-batch. Recompute total scheduled update count when changing `K`.

Interviewer expects: scheduler aligned with optimizer semantics.

Common mistake: Decaying the learning rate `K` times too quickly.

Q10. Does accumulation make training faster?

Answer: Its main goals are memory fit and effective batch size. It may reduce communication frequency but smaller kernels and extra serial micro-batches can lower throughput. Measure samples or tokens per second, not only update time.

Interviewer expects: nuanced systems trade-off.

Common mistake: Assuming fewer optimizer steps automatically means less total work.

7. Deep-Dive Questions

Q1. Prove equivalence for equal micro-batches.

Let each of `K` micro-batches contain `B` samples with mean loss $L_k = (1/B) \sum_i l_{ki}$. The combined mean is $L = (1/K) \sum_k L_k$. Differentiation is linear, so $\text{grad } L = (1/K) \sum_k \text{grad } L_k$. Accumulating `backward(L_k/K)` computes exactly that expression under fixed parameters.

Q2. How should token-level loss be normalized across unequal sequences and ranks?

Accumulate the sum of valid-token losses and the valid-token count. The desired gradient is the gradient of the global loss sum divided by the global token count. Because DDP may average across ranks, account for its reduction convention when scaling local numerators; do not average rank-local token means equally.

Q3. Why is BatchNorm nonequivalent under accumulation?

Each forward normalizes using only that micro-batch's mean and variance. A physical large batch would use statistics over all examples together, changing activations and therefore gradients. Accumulating the later gradients cannot retroactively combine the forward statistics.

Q4. How does accumulation affect Adam?

With one step after K micro-batches, Adam updates its first/second moments and bias-correction time index once using the combined gradient. Stepping Adam on every micro-batch performs K moment updates and parameter changes, so it is a different trajectory even if learning rate is divided.

Q5. How should overflow be handled in distributed mixed precision?

After the full window, unscale/check accumulated gradients. If any rank has nonfinite values, ranks must collectively agree to skip the optimizer update so replicas remain synchronized. Clear invalid gradients and update the loss scale according to the scaler policy; partial micro-batch rollback is generally not available.

8. Comparison Tables

Accumulation versus a physical large batch

Aspect	Physical batch	Accumulated micro-batches
Peak activations	Higher	Lower
Optimizer updates	One	One
Additive loss gradient	Reference	Equivalent with correct scaling
BatchNorm statistics	Whole batch	Per micro-batch
Kernel sizes	Larger	Smaller
Communication with DDP	Once per backward	Can be once per window with <code>no_sync</code>

Accumulation versus activation checkpointing

Aspect	Gradient accumulation	Activation checkpointing
Mechanism	Smaller micro-batches, sum gradients	Discard and recompute activations
Main purpose	Effective batch and activation peak	Activation peak
Extra work	More serial forward/backward calls for same update batch	Repeated forward work for same samples
Optimizer cadence	Deliberately delayed	Unchanged
Equivalence trap	Batch-dependent operations/loss scaling	RNG and side effects

9. Common Mistakes

- Calling `zero_grad()` after every micro-batch.
- Calling `optimizer.step()` after every micro-batch.
- Forgetting to normalize equal micro-batch mean losses by K .
- Dividing by K when actual valid counts are unequal.
- Stepping an update-based scheduler per micro-batch.
- Clipping before accumulation finishes.
- Unscaling mixed-precision gradients repeatedly or at the wrong boundary.

- Allowing DDP to all-reduce every micro-batch unnecessarily.
- Disabling DDP synchronization for the final backward too.
- Claiming exact equivalence despite BatchNorm or contrastive cross-sample losses.

10. Edge Cases / Special Cases

- **Partial final window:** normalize by its real size and decide whether to step or drop consistently.
- **Uneven ranks:** all ranks must participate in compatible final synchronization and weighting.
- **Variable tokens:** use valid-token totals rather than sequence counts.
- **Gradient penalties:** some algorithms call backward or inspect gradients inside a micro-step and need special handling.
- **Multiple optimizers:** each may have a different update frequency; clearing and scheduling must be explicit.
- **Sparse gradients:** accumulation semantics and buffer layouts may differ from dense gradients.
- **Truncated backpropagation:** detaching hidden state changes what “one effective batch” means.
- **Pipeline parallelism:** pipeline micro-batches and accumulation windows must agree on when the batch is complete.
- **EMA weights:** update exponential moving averages on optimizer steps, not micro-batches, unless intentionally defined otherwise.

11. How to Explain in Interview

Gradient accumulation runs several small micro-batches, adds their gradients into the same buffers, and calls the optimizer once. With local micro-batch B , data-parallel size N , and K accumulation steps, the effective batch is $B \cdot N \cdot K$. For equal mean losses I divide by K , synchronize DDP only on the final backward, then unscale, clip, step, schedule, and clear gradients once.

12. Quick Revision Notes

- **Pattern:** zero $\rightarrow$ K forward/backwards $\rightarrow$ clip $\rightarrow$ step $\rightarrow$ zero.
- **Effective batch:** $B \times DP \times K$.
- **Equal mean losses:** divide each by K .
- **Unequal counts:** weight by real samples/tokens.
- **DDP:** skip intermediate synchronization; synchronize final backward.
- **Optimizer/scheduler:** advance once per window.
- **Trap:** combined gradients do not create combined BatchNorm statistics.

13. Practice Tasks

1. Compare gradients from a batch of eight with four accumulated micro-batches of two.
2. Introduce unequal final micro-batch sizes and fix the normalization.
3. Count DDP all-reduces with and without a no-synchronization context.
4. Add mixed-precision scaling and clip only the final unscaled gradient.
5. Demonstrate why BatchNorm outputs differ between one large batch and accumulated micro-batches.
6. Recalculate training updates and scheduler warmup after changing K from 1 to 8.
7. Implement token-count-weighted accumulation for padded language-model batches.
8. Handle an epoch remainder of fewer than K micro-batches without under-scaling it.

14. Final Cheat Sheet

Item	Gradient-accumulation answer
Core definition	Sum gradients over micro-batches, then update once
Why it matters	Achieves a large effective batch under activation-memory limits
Effective batch	Local micro-batch x DP world size x accumulation steps
Correct scaling	Divide equal means by K ; weight unequal batches by valid count
Distributed optimization	Synchronize only at the final backward when supported
Most asked ordering	Accumulate -> unscale -> clip -> optimizer/scheduler step -> clear
Main trap	Not exactly a physical large batch for batch-dependent operations
One-line answer	"Gradient accumulation emulates a larger batch by adding several micro-batch gradients before one optimizer step."